TECHNICAL DOCUMENTATION

Jaddar Suite

Application Guide

Administrator and operator reference for Jadmin, Jaremin, Jareg and Jaddar, and the protocols that connect them

IN THIS GUIDE

Jadmin	Requestor Manager — requestor groups, users, subscriptions and their credentials
Jaremin	Group Admin — data holder groups, agreement templates and terms, subscription review
Jareg	Data Holder — RDAP data, policy and redaction, request review
Jaddar	Query application — where requestors look data up
Protocols	Standards the suite speaks: RDAP, OAuth 2.0 and OpenID Connect, request signing, ICANN RDRS and the rest

Derek Jenkins · October 2026

Contents

Overview

The four applications
How the applications connect

1. Jadmin — Requestor Manager

1.1 Dashboard
1.2 Subscriptions
1.3 Requestor Groups
1.4 Data Holder Groups
1.5 Users
1.6 Settings (Master Admin only)
1.7 My Profile

2. Jaremin — Data Holder Group Admin

2.1 Dashboard
2.2 Subscriptions
2.3 Templates
2.4 Data Holders
2.5 Applications
2.6 Requestor Groups
2.7 Data Holder Groups (master accounts only)
2.8 Instances (master accounts only)
2.9 Users (master and admin accounts)
2.10 Audit Logs (master accounts only)
2.11 Settings (master accounts only)

3. Jareg — Data Holder

3.1 Dashboard
3.2 Pending Requests
3.3 Data Holder Groups
3.4 RDAP Data
3.5 RDAP Query
3.6 Policy
3.7 Logs
3.8 Settings (Group Admin)
3.9 External Database
3.10 File Automation
3.11 Users (master accounts only)

4. Jaddar — Query Application

4.1 Dashboard (Home)
4.2 Requests
4.3 Data Holders (administrators only)
4.4 Logs (administrators only)
4.5 Settings (administrators only)

5. Protocols and Standards

5.1 Registration data (RDAP)
5.2 Sign-in and tokens (OAuth 2.0, OpenID Connect, JWT)

5.3 Connections between the applications
5.4 Transport and web security
5.5 ICANN RDRS and email
5.6 Data stores, files and formats
5.7 Connections at a glance

This guide describes the suite through its user interface, application by application. Each page section opens with the abbreviations it uses and the business concepts behind it, then names the on-screen controls in the order you would use them. The last part lists the protocols and standards the suite relies on and where each one is used.

Overview

The four applications

ABBREVIATIONS

RDAP	Registration Data Access Protocol
RDRS	Registration Data Request Service (ICANN)
ICANN	Internet Corporation for Assigned Names and Numbers

CONCEPTS

Requestor side	Jadmin and Jaddar serve requestor groups. They share one set of Keycloak accounts: the users and requestor groups created in Jadmin are the identities that sign in to Jaddar.
Data holder side	Jaremin and Jareg serve data holders. Each keeps its own administrator accounts, separate from Keycloak and from each other.
Requestor group	The organization requestors belong to, such as an agency. It is identified by its group code and subscribes to agreements on behalf of its members.
Data holder group	A set of data holders that offer agreements together, administered in Jaremin. Requestor groups subscribe to a data holder group's agreements, not to individual data holders.
Agreement	What the UI calls a template: the terms a data holder group offers requestor groups. Its request types decide what a requestor may ask for and at which access level.

- **Jadmin** — the Requestor Manager. Requestor groups, their users and member information, and their subscriptions to data holder group agreements.
- **Jaremin** — the Data Holder Group Admin. Data holder groups, agreement templates and their legal terms, subscription review and testing, and the credentials the other applications use to reach it.
- **Jareg** — the Data Holder. RDAP data, policy and redaction, request review, and the connection to a group admin.
- **Jaddar** — the query application. Where requestors actually look data up, including ICANN RDRS requests.

How the applications connect

CONCEPTS

Credentials	Every connection between the applications uses a Client ID and Client Secret that one application issues and an administrator enters in the other. Jaremin issues all three kinds: one per data holder (entered in Jareg), one per accepted requestor group (entered in Jadmin as Group Admin Credentials), and one per subscription (entered in Jadmin on the subscription).
Request signing key	A key pair Jadmin generates for each subscription. Jadmin signs its calls about the subscription with the private key, and Jaremin checks them against the public key registered there.
Token introspection	How a data holder checks a requestor's bearer token: it sends the token to the subscription's introspection URL, using introspection credentials that Jadmin delivers to the data holder group automatically. When an agreement requires member information, the answer also carries the member's values.
Subscription	A requestor group's acceptance of one agreement from one data holder group. It is reviewed in Jaremin, tested against a data holder's test data, and only an active subscription with credentials can be used in Jaddar.
Pinned terms	The copy of the agreement's terms a subscription was accepted under. Edits to the template reach a subscription only when the requestor group accepts a proposed or required change.

Each end of the workflows below is documented in its own application's section:

- **A data holder joins a group** — Jaremin registers the data holder or approves its application and issues credentials (2.4, 2.5); the data holder enters them in Jareg under Settings → Group Admin (3.8) and then sees its groups (3.3). Records marked as test data in Jareg (3.4) are what subscription tests query.
- **A requestor group is accepted by a data holder group** — Jaremin accepts the requestor group and issues its credentials (2.6); Jadmin enters them as the data holder group's **Group Admin Credentials** (1.4).
- **A requestor group subscribes to an agreement** — Jadmin submits the request, accepts the agreement's numbered legal terms and maps any required member information (1.2); Jaremin approves it, which tests it and activates it when the tests pass (2.2). Jaremin issues the subscription's own credentials, which are entered in Jadmin, and registers the signing key Jadmin generates (1.2, 2.2). Only then can Jaddar use the agreement (4.1).
- **An agreement changes** — the subscription keeps the terms it agreed to. Jaremin proposes the new terms or requires them by a deadline (2.3), and the requestor group accepts or declines in Jadmin (1.2).

> **Suggested setup order:** create data holder groups and users in Jaremin → register the data holders and connect them in Jareg → import policy, load data and mark test records in Jareg → build agreement templates in Jaremin → create requestor groups in Jadmin and accept them in Jaremin → subscribe from Jadmin, approve in Jaremin, and exchange the subscription credentials and signing key → query from Jaddar.

1. Jadmin — Requestor Manager

Navigation: Dashboard, Subscriptions, Requestor Groups, Data Holder Groups, Users (Requestor Admin and above), and Settings (Master Admin only). The header also shows the Jadmin version under the name, your name with your role badge, and **Logout**. Click your name to open a menu with **My Profile** (1.7); on a phone, tap your name at the top of the menu.

Sign in with your Keycloak email and password; **Forgot password?** sends a reset link. The sign-in page also shows the Jadmin version. After 15 minutes without activity (the default) the session ends. Two minutes before that, a *Still there?* dialog counts down with **Stay signed in** and **Sign out**. Activity in any open Jadmin tab keeps the session alive. When a session ends you return to the sign-in page with "Your session has expired. Please sign in again."

1.1 Dashboard

ABBREVIATIONS

RDAP	Registration Data Access Protocol

CONCEPTS

Role hierarchy	Jadmin has four roles, highest first: *Master Admin*, *Group Admin*, *Requestor Admin* and *Requestor User*. Each role can do everything the roles below it can, and your badge shows your highest role.
Group membership	You belong to a requestor group by being a member of the Keycloak group with the same name. Master Admins and Group Admins work across every requestor group; Requestor Admins and Requestor Users see and act only on the groups they belong to.
Requestor Group Administrator	Shown as *Requestor Admin*. Creates Requestor User accounts in their own groups, sets those groups' joining links and member fields, and looks after each subscription's credentials, signing key and member field mapping. Submitting, cancelling, testing and activating subscriptions are offered to Group Admins and above.
Requestor	A person who looks up registration data through Jaddar as a member of a requestor group, holding the *Requestor User* role here. Jaddar signs them in with the same account that Jadmin manages, and they keep their own details up to date in **My Profile**.

- A greeting with your first name, then "Welcome to Jadmin. You are logged in as", your role badge, and your groups.
- Counters, each linking to its page:
  - **Subscription Requests**, with the number that are active.

  - **Needs Your Action**: subscriptions that are approved or in testing, plus proposed agreement changes that are waiting for your response. It reads "All caught up" when there are none.
  - **Requestor Groups**, with how many have an active subscription.
  - **Users**, shown to Requestor Admin and above.
- **Subscription Requests by Status** is a doughnut chart of your requests by status. **Subscriptions by Requestor Group** is a stacked bar for each of the eight busiest groups, broken down by status. Its tooltip also counts proposed changes that are awaiting your response.
- **Quick Actions**: **View Groups**, and **Manage Users** for Requestor Admin and above.
- **Your Permissions** shows your **Current Role** with what it allows, and your **Group Membership**:
  - *Master Admin*: full system access across Jaddar and Jadmin.
  - *Group Admin*: can create and manage requestor groups.
  - *Requestor Admin*: can create users in assigned groups.
  - *Requestor User*: can view groups and make RDAP requests.

1.2 Subscriptions

"Manage your agreement subscription requests and active agreements"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
L0–L3	Access levels 0 (Public) to 3 (Full)

CONCEPTS

Subscription	A requestor group's acceptance of one agreement from one data holder group. The data holder group reviews it, it is tested, and only an *Active* subscription can be used for queries. Your group can hold one subscription per data holder group that is Draft, Submitted, Pending Review, Approved, Testing or Active; once it is Declined, Cancelled, Suspended or Expired you can request another. The limit counts the data holder group that owns the template, so when one group admin hosts several data holder groups you can hold one subscription with each of them.
Pinned terms	The copy of the agreement's terms the subscription was accepted under. When the data holder group edits its template, your subscription keeps running on the pinned copy; it moves to new terms only when you accept a change.
Proposed change / Required change	A proposed change is optional: decline it and the subscription stays exactly as it is. A required change carries a deadline, and if you have not accepted it by then the subscription is terminated.

Credentials	The data holder group issues a Client ID and Client Secret for each subscription in Jaremin, and you enter them here; calls about the subscription carry them, and without them only status checks work. Introspection credentials go the other way: Jadmin creates them when the subscription activates and delivers them, with Keycloak's public introspection address, to the data holder group itself (retried hourly, and straight away when you save credentials), so nobody types them. A third pair, the Group Admin Credentials, sits on the data holder group record (1.4).
Request signing key	A key pair for each subscription. Jadmin signs its calls about the subscription with the private key, which never leaves the server, and the data holder group checks them against the public key you register in Jaremin. Member information is verified only through signed calls, so it cannot be checked until the key is generated and registered.
Subscription test	Before activation the data holder group tests the subscription: validation checks, RDAP lookups under its request types, and, when the agreement requires member information, the details of the member who started the subscription. Jadmin starts the test and activates on a pass by itself; a failure stops for manual review, while a test waiting to verify member information is re-run automatically.
Member field mapping	When an agreement requires information about each member, your group chooses which member field supplies each item: a standard profile field or one of the group's own fields (1.3). Data holders receive the mapped values when they verify a member's request, and a member whose value is empty, or whose item is not mapped, is refused under that agreement until it is put right.

Subscribe to a template

1. Click **New Subscription**. In **Subscribe to a data holder group**, pick the group; only active groups are listed, and you can search by name or code. The same flow also starts from **Templates** on a data holder group card (1.4).
2. In **Subscribe to {group}**, choose which of your requestor groups the subscription is for. Master Admins and Group Admins see every requestor group; everyone else sees only the groups they belong to. The group's details prefill the request.
3. **Templates from {group}** shows "Subscribing as {requestor group}"; **Change requestor group** goes back a step. Each template is listed as described in 1.4. Click **Subscribe** on the one you want.
4. Complete the **Subscribe to: {template}** form:
   - The banner repeats the template's request types with their access levels. The **Data holder group contact** card shows who to contact at the data holder group about this agreement.
   - **Requestor Group** is fixed to the group you picked.
   - **Contact Information**: **First Name**, **Last Name**, and **Email** are required and are prefilled from your account. **Organization** and **Phone** are optional.
   - **Address Information**: **Street Address**, **City**, **State/Province**, **Postal Code**, and **Country**, prefilled from the requestor group's address.

  - **Data Holder Group Fields**: the data holder group's legal terms, set out as numbered sections and clauses (1, 1.1, 1.2 ...) with cross-references shown as clause numbers. Tick **I accept this section** for each one. Linked documents need **I have reviewed this document** where they are required.
  - **Member Information**, shown only when the template requires information about each member who makes requests: for each required item, choose the member field that supplies it from **Standard fields** or **This group's fields**. Jadmin preselects the standard field with the same name, or a group field with the same name. Every item must be mapped before you can save or submit.
  - **Requestor Group Fields**: the answers the template asks your group to supply. Each is a text, paragraph, number, date, select, or checkbox field; required fields are marked, and limited text fields show a character counter. Last comes **Token Introspection URL**, which is required and prefilled from the requestor group's default. Without it the data holder cannot accept any query from your group.
  - **Submit immediately to the data holder group**: untick it to keep the request as a draft. The button then changes from **Submit Request** to **Save as Draft**.

> **Limit:** a requestor group can have only one subscription with each data holder group. A second request is rejected with a message that names the data holder group and points you to the existing subscription. The limit applies to the group that owns the template, not to the group admin hosting it: when one entry under Data Holder Groups (1.4) offers templates from several data holder groups, you can subscribe to one template from each.
>
> **Note:** if you belong to no requestor group, the group picker shows "You are not a member of any requestor groups", and you cannot go on to subscribe.

Tracking requests

- Summary tiles:
  - **Total Requests**.
  - **Pending**: drafts, plus requests that are submitted or pending review.
  - **Action Required**: requests that are approved or in testing.
  - **Active Agreements**.
- An **Action Required** banner splits the Action Required count into requests ready for testing and requests in testing.
- **Active only** narrows the list to active subscriptions. A badge beside it counts the requests that need attention.
- Filters: a search box, a status filter, a requestor group filter, and **Refresh**. The page re-checks in-flight requests (*Submitted*, *Pending Review*, *Approved*, *Testing*) every 5 seconds and shows when it last checked. A notification announces each status change.

- The table lists Request ID, Template, Data Holder Group, Requestor Group, Status, Requested By, and Created. The agreement ID appears under the Request ID once one is assigned. Under the status you see:
  - a credentials badge, *Credentials received* or *No credentials*;
  - a progress hint for in-flight requests;
  - any change the data holder group has proposed.
- Row actions:
  - **View Details**.
  - **Submit**, for drafts.
  - **Refresh Status**, for in-flight requests.
  - **Cancel**, for requests in Draft, Submitted, or Pending Review. You confirm with **Cancel Request**, and it cannot be undone.
- While testing and activation run automatically, the row shows an indicator ("Automated testing & activation in progress") in place of the manual controls. **Submit**, **Cancel**, and the testing controls are for Group Admins and above.

Lifecycle

Draft → Submitted → Pending Review → Approved → Testing → Active. Declined, Cancelled, Suspended, and Expired are terminal or paused states.

- While the data holder group reviews a request, the hint reads "Awaiting review…" (Submitted) and then "Under review…" (Pending Review).
- Once the data holder group approves, Jadmin moves the subscription on by itself, one step at a time, whether or not anyone has the page open:
  - testing starts ("Testing starts automatically…");
  - the tests run ("Automated testing in progress…");
  - if they pass, the subscription activates ("Tests passed — activating…").
- If testing does not pass, the subscription stays in Testing with "Testing did not pass — manual review needed". **Run Tests** then appears on the row, and **Activate** is added once a run passes.
- If everything passed except the member information check, the subscription stays in Testing and Jadmin re-runs the test by itself. When credentials or a signing key are still missing, the status message says which ("Testing: PENDING — member information cannot be verified yet: …"). The test re-runs as soon as you save credentials or generate keys, and at most every 5 minutes once both are in place; the subscription activates when it passes.

Subscription details

View Details opens **Subscription Request Details**:

- **Status banner**: the status, when it last changed, the status message, and a next-step hint. For Group Admins and above it also offers **Start Testing** when the request is Approved, or **Run Tests** and **Activate** when it is in Testing. Until the tests pass, it reads "Run tests and wait for them to pass before activating".
- **Request data** (collapsed by default):
  - **Request Information**: Request ID, External ID, Template, Data Holder Group, Requestor Group, Request Types, Created, Submitted, and Expires.
  - **Contact Information**: Full Name, Organization, Email, and Phone.
  - The data holder group contact, exactly as the group publishes it today.
- **Member Information**, shown when the agreement requires information about each member: each required item with the member field that supplies it, or "Not mapped — members cannot use this agreement". Requestor Admins and above can change the choices and click **Save Mapping**. Changes take effect on the next request.
- **Agreement terms**: "These are the terms this subscription runs under, as at {date}. They do not change unless you accept an update."
  - **Data Holder Group Fields** lists each numbered section with the date you accepted it, and each linked document marked *Reviewed*.
  - **Requestor Group Fields** shows the answers you gave.
  - **Request types available** lists each type with its access level (L0–L3).
- **Data Holder Group Credentials**: described under Credentials below.

Credentials

- The data holder group issues a client ID and secret for each subscription. Enter them in **Client ID** and **Client Secret** and click **Save credentials**. When a secret is already stored, leaving the field blank keeps it. You can enter credentials at any status. If the agreement requires member information, enter them and register the signing key before testing finishes, because the test checks member information over a signed call.
- The banner reads *Credentials received* or *No credentials*. Until credentials are entered, status checks still work, but every other call to the data holder group is refused and the agreement cannot be used for queries. When the data holder group refuses a call, the error says whether no credentials have been entered or whether it did not accept the credentials or signing key entered.
- Saving credentials on an active subscription also delivers its introspection credentials to the data holder group at once if they had not yet arrived.
- **Request Signing Key**: **Generate keys** (**Regenerate keys** once keys exist) creates a key pair. You confirm first in **Regenerate signing keys?**, which warns that calls fail until the new public key is registered with the data holder group. Copy the **Public key** and give it to the data holder group to register. The private key never leaves the server.

- Saving credentials and generating keys require Requestor Admin or above. On a subscription in Testing, either one re-runs the test.

Proposed changes

A subscription stays on the terms it agreed to, even when the data holder group changes its template. To move a subscription onto new terms, the group proposes a change, which appears under the status in one of two forms:

- *The data holder group has proposed a change* is optional: "Your subscription is unchanged until you respond. Decline and it stays exactly as it is."
- *A required agreement change needs your acceptance* has a deadline: "You must accept by {date}. If you do not, this subscription will be terminated."

To respond:

1. Click **Review the changes and respond**. The details open as **Proposed agreement terms — {template}**.
2. Tick **I accept this section** for every section and **I have reviewed this document** for each required document. Answer every required requestor group field, including any new ones. Until everything is complete, the footer reads "Accept each section and answer every required field to continue."
3. Click **Accept change**. For an optional change you can click **Decline** instead; the status then shows "You declined a proposed change; this subscription is unchanged."

Test results

After **Run Tests**, the **Test Results** window reads *All Tests Passed!*, *Some Tests Failed*, or *Waiting to Verify Member Information*. It shows:

- a summary: **Validation Tests** passed and failed, **RDAP Tests** passed, failed, skipped, and errored, and total **Duration**;
- the Request ID and when the tests ran (**Tested At**);
- the list of validation tests, each marked *PASSED*, *FAILED*, or *Pending*;
- **Member Information Sent**, when the agreement requires it: the details of the member who started the subscription that the data holder group verified, with *Not filled in* for any missing value;
- **RDAP Data Tests**, with one card for each lookup showing the request type, access level, duration, and checks passed.

RDAP checks are grouped as Lookup, Access, Parameters, Contact Visibility, and Redaction. Each check shows a severity and its expected and actual values. When every test passes, **Activate Subscription** finalises the agreement.

> **Note:** *Waiting to Verify Member Information* means everything else passed. Once the subscription's data holder group credentials are entered and its signing key is registered with the data holder group, the test runs again by itself and the subscription activates when it passes.

1.3 Requestor Groups

"Manage organizations and groups that can request data access"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
ICANN	Internet Corporation for Assigned Names and Numbers

CONCEPTS

Requestor group	The organization requestors belong to, such as an agency. Saving one creates a Keycloak group of the same name if none exists, renaming it renames the Keycloak group, and membership of that Keycloak group is what makes someone a member. Its address prefills the group's subscription requests.
Group code	Data holders identify your group by its code and use it to match each RDAP query to the right subscription, so a group cannot be created without one. Codes are unique and stored in uppercase.
Token introspection	A data holder checks each requestor's bearer token by sending it to the subscription's introspection URL, signing in with the introspection credentials Jadmin delivered for that subscription. When the agreement requires member information, the answer also carries the member's values and lists any that are missing. Without a URL every query from your group is refused, so it is required on the group and on each subscription.
Requestor member fields	Information your members fill in on their profile beyond the standard details (name, email, phone number and address). Each field is Text, Number, Date or Choice, its name must be unique and differ from the standard fields, and removing a field also deletes every member's value for it. Agreements that require member information are mapped to these fields or to standard ones (1.2).
Joining link	An optional web address where someone outside every requestor group can apply to join yours. Data holders are never given the link itself, only a Jadmin address that redirects to it, so a changed link takes effect at once and a removed one stops being offered.
Public Information Services	A data holder that offers requestor group links lists, in its answers to public lookups, the joining links of the requestor groups with an active subscription it serves. It reads each link through token introspection, so your group appears only once it has an active subscription whose introspection credentials have been delivered.
Who can change a group	Master Admins and Group Admins can change every detail. A Requestor Admin can change only the joining link and member fields of their own groups, because data holders identify and authorise the group by its name, code and introspection URL.

Master Admins and Group Admins see every group and can create, edit, and delete them. Everyone else sees only the groups they belong to. A Requestor Admin can also edit their own groups, but only the joining link ("As a requestor group admin you can change this group's joining link. The other details are managed by your administrators."), and manage their member fields.

Create a group

1. Click **New Group**.
2. Complete **New Requestor Group**. Every field except Description and Joining Link is required.
   - **Group Name** should match a Keycloak group name for user membership. Saving a new group creates that Keycloak group if it does not already exist, and renaming the group renames it in Keycloak too.
   - **Group Code**: a short unique identifier of 10 characters or fewer, made of letters, digits, and hyphens, that cannot start or end with a hyphen. It is converted to uppercase as you type. Data holders identify the group by this code, and it is used in RDAP query parameters. A *Valid* indicator confirms the format.
   - **Description**.
   - **Default Introspection URL**: the OAuth 2.0 token introspection endpoint for the group. It is filled in with the public introspection address that this deployment's Keycloak advertises in its own configuration, so data holders on other servers can reach it; edit it if your endpoint differs. If Keycloak advertises only a localhost address, or cannot be read, the internal address is used instead, which data holders outside this server cannot reach. It prefills new subscription requests so that data holders can validate bearer tokens from your members.
   - **Address Information**: **Street Address**, **Country**, **City**, **State / Province**, and **Postal Code**. Country is a searchable picker grouped by ICANN region. The address prefills subscription requests made for this group.
   - **Joining**: **Joining Link**, where someone can apply to join the group, such as your application form. It must be a full web address starting with https:// or http://, up to 500 characters. When editing a group that has a link, the form also shows the address that data holders hand out.
3. Click **Create**.

Member fields

Member fields are edited from the group's edit form, by Group Admins and above and by Requestor Admins of that group. A new group gets its member fields once it has been created.

1. Click the edit button on the group card. At the bottom of the form, under **Member Fields**, click **Edit Member Fields** to open **Member Fields: {group}**.
2. Click **Add Field** for each item your members should fill in on their profile.
3. Enter the **Field name** and choose the **Type**: Text, Number, Date, or Choice. For Choice, list the **Choices** separated by commas. **Help text** is optional and appears under the field on the profile.
4. Use the delete button to remove a field; members' values for it are deleted when you save.
5. Click **Save**. Saving or cancelling returns you to the group's edit form, with any changes you had not yet saved there still in place.

Review a group

- Each card shows the group's name, ID, code, description, creator, and creation date. Search covers name, code, and description; **Clear** resets it. The list is paginated.
- **View Details** opens **Group: {name}**:
  - Code, description, and **Default Introspection URL** ("Not configured — will need to be set per subscription" if there is none).
  - **Joining Link**, with the "Public link given out by data holders:", or "No joining link. Data holders will not list this group in public answers."
  - Address ("No address on file." if there is none), and who created the group and when.
  - **Members**: each member's name, email, and highest role.
  - **Active Agreements**: name, data holder, and status.
  - **Subscription Requests**: template, data holder, status, and request date.
- Deleting a group asks you to confirm and warns that it "will also affect all associated agreements."

1.4 Data Holder Groups

"Manage registered data holder groups and subscribe to their agreement templates"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
API	Application Programming Interface

CONCEPTS

Data holder group	A set of data holders that offer agreements together, administered in Jaremin. Registering it here points Jadmin at that group's API by its code and base URL, and only active groups can be subscribed to.

Group Admin Credentials	The Client ID and Client Secret the data holder group issues in Jaremin when it accepts your requestor group, entered here on the group record. Jadmin presents them so the group's API knows which requestor manager is calling; calls about one subscription present that subscription's own credentials instead, once they are entered (1.2).
Agreement	What the UI calls a template: the terms a data holder group offers requestor groups. It bundles request types, daily and monthly query caps, the group's legal sections, the fields your group must answer, and any information each member must supply.
Request type	One kind of request allowed under an agreement. It sets the access level, which RDAP data the request returns, any custom parameters, and its flags.
Access level	Level 0: Public, Level 1: Basic, Level 2: Enhanced or Level 3: Full. The higher the level, the more registration data a request may see once the data holder's own redaction rules are applied.
Confidential, Exigent, Manual approval	A Confidential type lets a requestor ask that the registrant is not notified of the lookup. An Exigent type is for emergencies: an exigent request skips the manual review it would otherwise wait for. *Manual* means each request of that type waits for someone at the data holder to approve it; a Standard type is neither confidential nor exigent.
Data holder group fields / Requestor group fields	Data holder group fields are the legal sections and clauses you must accept, plus linked documents to review. Requestor group fields are the answers your group supplies. Both are pinned with the subscription when you subscribe.

Register a group

Group Admins and above:

1. Click **Add Data Holder Group**.
2. Enter the **Code** (uppercase letters, numbers, underscores, and hyphens, e.g. ARIN; it cannot be changed later), the **Name**, and the **Base URL** of the group's API without the */api/...* path.
3. Under **Group Admin Credentials**, enter the **Client ID** and **Client Secret** that the data holder group issued when it accepted your requestor group. They identify this Requestor Manager to that group admin. When you edit the group later, leaving the secret blank keeps the stored one.
4. Optionally add a **Description**, **Contact Email**, and **Notes**. Leave **Active** ticked, then click **Create**.

Working with a group card

- Each card shows the code and name, then badges for *Active* or *Inactive* and for health: *Healthy*, *Unhealthy*, or *Unknown*.
- Below that are the description, base URL, contact email, last contact time, and *API Key configured* where one is set.
- A source badge says where the entry came from: *Entered here*, or *Central repository* for a group added automatically from the common repository (1.6). A repository entry is kept up

to date by the repository and cannot be edited here, because edits would be overwritten. *Duplicate code* marks two entries claiming the same code; both are listed, so check which one you mean before subscribing.

- **Templates** starts the subscribe flow (1.2). Once you pick a requestor group, **Templates from {group}** lists each published template with:
  - its description and ID;
  - its request types, each marked Standard, Confidential, or Exigent, with its access level (Level 0: Public to Level 3: Full) and *Manual* where manual approval is needed;
  - the group types it requires;
  - its daily and monthly query caps.
- **View full terms and fields** expands a template to show:
  - the data holder group contact and the full request type descriptions;
  - **Data Holder Group Fields**: the numbered legal terms and linked documents;
  - **Requestor Group Fields**: what your group will have to fill in, with each field's type, whether it is required, and any character limit.
- **Health** runs a connectivity check and reports "{name} is healthy" or "{name} is unhealthy: {error}".
- Group Admins and above can edit a group and **Deactivate** or **Activate** it. Master Admins can also delete it; you confirm first, and it cannot be undone.
- Search, a status filter (Active, Inactive, Healthy, Unhealthy), **Clear**, and **Refresh**. The list is paginated.

1.5 Users

"Manage user accounts and group memberships"

CONCEPTS

User account	Each user is a Keycloak account, the same one used to sign in to Jaddar. An email that already has an account is added to your group rather than creating a second account.
Groups and role	A user's groups decide which requestor groups they act for, and their role decides what they can do there.
Who can manage whom	You can create, edit and assign roles only below your own role, and Master Admin accounts cannot be created or granted here. A Requestor Admin can create only Requestor Users, must place them in one of their own groups, and can edit users only within those groups. Deleting needs Group Admin or above, and you cannot delete your own account.

- **New User**: **Email** (required; it cannot be changed later), **First Name**, **Last Name**, **Password** (new accounts only), **Groups** (multi-select), and **Role**. Only roles below your own are listed, and Master Admin accounts cannot be created here.
- If the email already belongs to an account, the **User Already Exists** dialog shows that user's name, email, and current groups. It offers **Add to Group** instead of creating a duplicate.
- The **Users** table lists User, Email, Groups, and Role, with a search box, group and role filters, **Clear**, and a refresh button.
- You can edit only users whose role is below yours; a Requestor Admin can edit them only within their own groups. Master Admins can edit anyone.
- Editing a user also shows their **Member Profile**, the same information they keep in My Profile (1.7):
  - the standard fields **Phone Number**, **Street Address**, **City**, **State / Province**, **Postal Code** and **Country** (the name and email are the fields above them);
  - **Requestor Group Member Fields**: the member fields of each requestor group the user belongs to, under the group's name, with the badges for the agreements that require each field. When none apply, it says the user belongs to no requestor group or that their groups have defined no member fields.
- Clicking **Update** saves the profile with the rest of the form. A Requestor Admin's changes to group member fields are saved only for the groups they share with the user. When you may view a member's profile but not change it, the section reads "You can view this member's profile but not change it."
- Group Admins and above can delete users whose role is below their own. You cannot delete your own account. Deleting asks you to confirm and cannot be undone.

1.6 Settings *(Master Admin only)*

"Deployment-wide settings for this Requestor Manager instance." The **Logo** tab holds the logo and login notices; the **Common Repository** tab connects Jadmin to a central repository of data holder groups.

Logo

- **Upload logo** replaces the default logo on the login screen and in the header. The file can be PNG, JPEG, GIF, or WebP, up to 2 MB; a wide image with a transparent background works best.
- The current logo is labelled *Your logo* or *Default logo*, with its file name, size, who uploaded it, and when.
- **Replace logo** swaps it; **Remove** goes back to the default.

Login notices

- Extra text shown under the sign-in form, such as a privacy notice, an acceptable-use statement, or a link to your terms. Nothing appears until you add an entry.
- Each notice has **Text** and **Link (optional)**. Add a link to turn the text into a hyperlink; leave it blank for plain text.
- **Add notice** creates an entry, up to 10. **Move up** and **Move down** set the display order, and **Remove** deletes an entry.
- **Save notices** saves the list.

Common Repository

Central data holder group repository: a common repository is where group admins (2.11) announce their data holder groups. With it turned on, the groups it publishes are added under Data Holder Groups (1.4) automatically, tagged *Central repository*.

- **Query the central data holder group repository** turns it on. While it is off, only data holder groups entered by hand are listed.
- **Repository feed URL**: the repository's feed address. Jadmin's server fetches it, not your browser, so it must be an address the server can reach.
- **Save** stores the settings. **Re-fetch the listing** reads the repository again at once; otherwise Jadmin re-reads it every 15 minutes.
- The panel shows when the listing was last checked and how many data holder groups it lists.

1.7 My Profile

"Your contact details and the information your requestor groups ask for. Data holders verify this information when you make requests under agreements that require it."

CONCEPTS

Standard member fields	First Name, Last Name, Email, Phone Number, Street Address, City, State / Province, Postal Code and Country, available to every requestor group's agreements. Your name is saved to your sign-in account, and your email is your sign-in name, so you cannot change it here.
Group member fields	The extra fields each of your requestor groups asks for (1.3), in a card per group. Values must fit the field's type: a number, a date (YYYY-MM-DD), or one of the listed choices.
Required by	A badge under a field names each agreement that draws on it: a subscription of your group, from Submitted through Active or Suspended, that maps a required item to that field. A highlighted badge means the field is empty, and data holders refuse your requests under that agreement until you fill it in and save.

Open it from the menu under your name in the header. Every signed-in user has one.

- **Personal Information**: **First Name** and **Last Name** (both required), **Email** (read-only: "Your email is your sign-in name; an administrator can change it."), **Phone Number**, **Street Address**, **City**, **State / Province**, **Postal Code**, and **Country**.
- **{group} Member Fields**: one card for each of your requestor groups that has member fields, with each field's help text under it. When none of your groups has any, a single **Requestor Group Member Fields** card says so: "You are not a member of any requestor group, so no group member fields apply to you." or "Your requestor groups ({groups}) have not defined any member fields."
- Administrators can see and edit the same information on the user's edit form in Users (1.5).
- Under a field, a badge names each agreement that requires it; hover over a badge to see the data holder group and which required item the field supplies. The footer explains: "Agreements listed under a field require it. A highlighted agreement cannot be used until the field is filled in."
- Click **Save Profile**.

2. Jaremin — Data Holder Group Admin

Navigation has a flat **Dashboard** plus three menus: **Agreements** (Subscriptions, Templates), **Directory** (Data Holders, Applications, Requestor Groups, and for master accounts DH Groups and Instances), and **Admin** (Users for master and admin accounts, Audit Logs, and Settings for master accounts). The header also has a group selector (shown as **Acting as** on narrow screens) that scopes what the pages show. Master accounts can choose **All Groups** or a single group. Other accounts choose among their assigned groups, and their first group is selected automatically. The header and the sign-in page show the Jaremin version number.

> **Note:** an idle session ends after a set time (15 minutes by default). Before it ends, a *"Still there?"* dialog offers **Stay signed in** or **Sign out**. Once the session has expired, the sign-in page reports *"Your session has expired. Please sign in again."*

2.1 Dashboard

ABBREVIATIONS

DH, DHs	Data Holder, Data Holders

"Jaremin Overview", with a badge for the group in scope and a **Refresh** button.

- Counter cards, each linking to its page: **Total Subscriptions** (with the number active), **Pending** (*Awaiting review* or *Nothing awaiting review*), **Total Templates** (with the number globally published), and **Data Holders** (with the number active).
- **Subscriptions by Status** is a chart of Pending, Approved, Testing, Active, Suspended and Denied subscriptions.
- **Templates & Infrastructure** is a chart splitting templates, data holders, DH groups (master accounts only) and instances (once any exist) into Active and Inactive.
- Master accounts also see **DH Groups**, **Active Groups** and **Total Users**.
- **Pending Subscriptions** lists up to ten, with Request ID, Requestor Group, Template, Data Holder Group and Created. **View All** opens the Subscriptions page.

2.2 Subscriptions

ABBREVIATIONS

RDAP	Registration Data Access Protocol

CONCEPTS

Subscription	A requestor group's acceptance of one of your agreements, submitted from Jadmin with every legal section accepted and the required requestor group fields answered. It belongs to the agreement's data holder group rather than to one data holder, so every data holder in the group honours it while it is ACTIVE. It moves from PENDING to APPROVED, TESTING and ACTIVE; only a pending one can be denied, only an active one suspended, and **Reactivate** returns a suspended one to ACTIVE without a new test.
Subscription test	Automated checks that the requestor group's identity and contact details are complete, that the agreement is still active with at least one active request type and valid access levels, that the group has an active data holder, and that the subscription has not passed its end date. A retrieval test then asks each active data holder in the group in turn, and the first one that holds *test data* for this agreement decides the outcome. When every check passes the subscription activates at once; when the only open check is the requestor member information that Jadmin has yet to supply, the result is PENDING rather than FAILED.
Pinned terms	The copy of the agreement's legal sections, requestor group fields, requestor member fields and request types that the subscription runs under, taken when the group subscribed. Editing the template never changes it, and data holders read this copy when they decide what a request may do. Only a published change that the requestor group accepts replaces it.
Proposed change / Required change	A change you publish from Templates (2.3) waits beside the pinned terms until the requestor group answers in Jadmin. A proposed change is optional and can be declined, leaving the subscription as it is. A required change cannot be declined, and if it is not accepted by its deadline the subscription is terminated and becomes CANCELLED.
Credentials	Each subscription has its own Client ID and Client Secret, issued here and entered by the requestor group in Jadmin. They authorize calls for that one subscription only: starting and running its tests, activating it, answering proposed changes, and delivering its introspection credentials. Regenerating deactivates the old pair at once.
Request signing key	An Ed25519 public key that Jadmin generates for one subscription and that you register here once credentials exist. Calls that Jadmin signs with the matching private key are accepted in place of the Client Secret, and only a signed test run can carry the member information that an agreement's requestor member fields require.
Token introspection	How a data holder checks the bearer token on each request with the identity service that signed the requestor in, at the **Introspection URL** recorded on the subscription. Jadmin also delivers a separate introspection client ID and secret for the subscription, which data holders use to call that URL; the answer identifies the requestor and carries the member information the agreement requires. Only the URL is shown here.

A subscription is a requestor group's agreement to one of your templates. Requestor groups submit them from Jadmin, and you review them here.

The list

- Filter by status: **ALL**, **PENDING**, **APPROVED**, **TESTING**, **ACTIVE**, **SUSPENDED** or **DENIED**. You can also search by request, requestor, data holder or template.

- Columns: Request ID, Requestor Group (with organization), Template, Data Holder Group (with the data holder, where one is recorded), Status, Test (PASSED, FAILED or PENDING once a test has run), and Created.
- Row actions depend on status: **View** always; **Approve** and **Deny** while pending; **Start Test** once approved; **Run Test** while testing, which opens the details; **Suspend** while active; **Reactivate** while suspended.

Approving and denying

1. Click **Approve** or **Deny**. The dialog shows the requestor group, where the subscription is in the workflow, and its current status.
2. **Deny** and **Suspend** need a **Reason**. For the other actions, **Notes (optional)** is optional.
3. Approval tests the subscription immediately and activates it when the tests pass. The result is reported as *"Approved — tests passed and the subscription is active"* or *"Approved, but the automatic tests did not pass — review the results"*. The second message also appears when the test is only waiting on Jadmin for member information (see Testing Phase).

Start Test is only needed when the automatic test did not run after approval. It moves the subscription into testing, and you can then run the tests from its details. Either side, admin or requestor, can start testing.

Subscription details

Click a Request ID or **View** to open the details, titled with the requestor group's name.

- A progress bar shows **Pending → Approved → Testing → Active**. If the subscription ends up DENIED, SUSPENDED, EXPIRED or CANCELLED, that status is shown in red instead. During testing a *Live* badge appears and the details refresh on their own.
- Guidance for the next step, and any template change waiting on the requestor. The page shows the proposal date and, for a required change, the date by which it *"Must be accepted by"*. It also shows when the requestor declined a change or let a required change expire.
- **Subscription**: Request ID, Template, Data Holder Group, Data Holder (when recorded), Purpose, **Introspection URL**, Effective From, Effective To, Created, and, where they apply, Request expires, Activated (with who activated it), Query limits and Status message. Master accounts can change the introspection URL in place with the pencil button.
- **Requestor Group**: Group, Group ID, Type, Contact, Email, Organization, Address, Phone, and, where they apply, Group code, Description and Callback URL.
- **Credentials**: see below.
- **Agreement terms in force**: see below.
- **Review**: who reviewed the subscription, when, and their notes.
- **Status History**: every status change, with who made it and why.

Credentials

Each subscription has its own credentials, which Jadmin uses to connect to this group admin for that subscription.

1. Click **Generate credentials**. **Client ID** and **Client Secret** appear with copy buttons. The secret is shown only once. After that, **Regenerate credentials** issues a new secret and replaces the old one.
2. Give both values to the requestor group, who enters them in Jadmin.
3. Under **Request Signing Key**, paste the public key that Jadmin generated for this subscription and click **Register**. The panel then shows *"A signing key has been registered"* and the date. Until a key is registered, the subscription cannot use signed requests.

You can only register a key after generating credentials. Regenerating credentials also removes the registered key, so register it again afterwards. Once credentials have been used, the panel shows when they were last used.

Agreement terms in force

These are the terms the subscription runs under, pinned when it was agreed. Editing the template does not change them.

- **Data Holder Group Fields**: each numbered legal section with its clauses, marked *Accepted* with a date, or *Not accepted*. Subscriptions from before per-section acceptance was recorded say that the group accepted the agreement as a whole.
- **Documents for review**: each linked document, marked *Reviewed* or *Not acknowledged*.
- **Requestor Group Fields**: the answers the requestor group gave when subscribing.
- **Request types**: each request type with its Type Code and access level.

When a change is pending, **In force** and **Proposed** switch between the current terms and the proposed ones. The proposed view states whether the change is optional, or required with a deadline.

Testing Phase

The **Testing Phase** panel appears while a subscription is in testing, and stays afterwards so you can review the results. Only this group admin sees it.

- A header badge reads *Awaiting test execution*, *Tests Passed*, *Tests Failed* or *Waiting on requestor manager*.
- While testing, **Run Test** (or **Re-run Tests** after a result) runs the tests again. Before any result exists, **Mark Passed** and **Mark Failed** set the result by hand. The subscription activates when the tests pass, including when you click **Mark Passed**.
- When a run has not passed, **Why the test has not passed** lists each check that did not pass with its reason. Checks still waiting on Jadmin are marked *(waiting)*.

- The result shows the summary, each check with its failure message and technical detail, and the **Started** and **Completed** times.
- **What was tested** expands to show which data holder ran the retrieval test (or *"No data holder in this group ran a retrieval test"*) and how many queries passed. It lists each RDAP query with its result (PASSED, FAILED, SKIPPED or ERROR), request type, access level, duration and per-check results with expected and actual values. When no query ran, it lists the attempts to reach a data holder. **Member information received** shows, for the member who started the subscription, each required member field with its value or *Not supplied*. **Waiting on the requestor manager** lists checks that only Jadmin can complete, and **Checks that did not pass** gives each failed check's reason and technical cause.

> **Note:** the retrieval check asks an active data holder in the template's group to run real RDAP lookups against its test data for that template. The test fails when the group has no active data holder, when none can be reached, or when none has test data for the agreement.
>
> **Note:** when the agreement requires requestor member information (2.3), a test you run here can at best end *Waiting on requestor manager*, because only Jadmin can supply that information, signed with the subscription's key. Once credentials are entered in Jadmin and the signing key is registered here, Jadmin runs the test again on its own, and the subscription activates when that run passes. The waiting check explains what is still missing: credentials, the signing key, or a correctly signed run.

2.3 Templates

"Agreement Templates"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
RDRS	Registration Data Request Service (ICANN)
ICANN	Internet Corporation for Assigned Names and Numbers
ASN	Autonomous System Number
DNSSEC	Domain Name System Security Extensions
L0–L3	Access levels 0 (Public) to 3 (Full)
Conf.	Confidential (request type flag)
Exig.	Exigent (request type flag)
Manual Appr.	Manual Approval (request type flag)
OCR	Optical Character Recognition
DPI	Dots Per Inch
JSON	JavaScript Object Notation

CONCEPTS

Visibility	How widely an active template is advertised. **Private** templates are listed nowhere and can only be reached by template ID or by existing subscribers, **Public** ones are listed when someone looks this data holder group up, and **Globally published** ones are also published for discovery beyond this group admin. The group's own visibility is a ceiling, a disabled group makes all its templates private in effect, and other groups can copy only active templates that are effectively Public or wider.
Request type	One kind of request allowed under the agreement. An **RDAP** request type sets the access level, the RDAP fields disclosed, the custom parameters the requestor must supply, an optional query pattern and the request flags, and its Type Code identifies it across the whole deployment. An **ICANN RDRS** request type instead sends a registration data request to ICANN's RDRS with the requestor's own ICANN credentials and never reaches a data holder.
Access level	How much registration data a request may see, from L0 Public through L1 Basic and L2 Enhanced to L3 Full. Each data holder applies the level to its own policy, so the same level can reveal different fields at different data holders, and fields switched off under **RDAP Params** are withheld whatever the level. A data holder can also be set to hold requests at or above a given level for manual review.
Confidential, Exigent, Manual approval	**Conf** lets requestors mark a request under this type as confidential, meaning the registrant is not to be told of the disclosure. **Exig** lets them mark it exigent (an emergency), which at the data holder skips manual review that would otherwise apply; **Manual Appr.** sends every request under this type to the data holder's Pending Requests for a person to approve, unless it is an exigent request the type allows. The presets set these for you: Standard is L1 with manual approval, Confidential L2 with Conf and manual approval, Exigent L3 with Exig and no manual approval.
Data holder group fields	The legal terms of the agreement, set out as numbered sections of clauses. A requestor group must accept every section to subscribe, and each acceptance is recorded with its date. A cross-reference remembers which section or clause it points at rather than its number, so the number shown in the agreement stays right when sections move.
Requestor group fields	Answers the requestor group gives once, when it subscribes. Required fields must be filled in, a URL field with **Acknowledgement required** must be confirmed as read, and Text and Paragraph answers must fit their character limit. If a published change adds a required field, the group must answer it when it accepts the change.
Requestor member fields	Information every member of the requestor group must have on their profile in Jadmin to make requests under this agreement, chosen from nine standard fields or added as custom ones. The data holder reads it from the member's token at request time and refuses the request, naming what is missing, when any field is empty. Before activation, the subscription test checks the member who started the subscription, which only a test run that Jadmin signs with the subscription's key can supply.

A template is the agreement a requestor group subscribes to. Its **request types** are the kinds of request allowed under it. Each one sets an access level, the RDAP fields disclosed, and any extra data the requestor must supply.

The list

- Counters: **Total**, **Active (page)**, **Inactive (page)** and **Global (page)**. You can search by name or ID.
- Columns: Template ID, Name (with short description), Group (in All Groups mode), Request Types (a Standard, Confidential or Exigent badge, the access level, and the number of custom parameters), Intended Requestors (*Any* when none are set), Status, Visibility, and Actions.
- Click the **Active** / **Inactive** badge to toggle it. An active template accepts new subscriptions. Deactivating one refuses new subscriptions but leaves existing ones as they are.
- The **Visibility** dropdown sets how widely the template is advertised:
  - **Private**: not listed anywhere. Only someone who already knows the template ID, or who is already subscribed, can reach it.
  - **Public**: listed when someone looks this data holder group up.
  - **Globally published**: listed like Public, and also published for discovery beyond this group admin.
- A template can never be advertised more widely than its group allows (2.7). When the group's setting lowers it, a warning badge shows the level that actually applies.
- Row actions: **Publish changes to subscribers**, **Edit** and **Delete**.

> **Limit:** a template with active subscriptions cannot be deleted.

Create or copy a template

- **New Template** opens the editor for the group you are working in.
- **Copy agreement** lists your own group's agreements, plus the active agreements that other data holder groups advertise as Public or Globally published. Each entry shows its group, visibility, and counts of request types, legal sections and requestor fields. Click **Copy** to add a copy to your group. The copy is named "(copy)", starts private and inactive, uses your group's contact, and opens in the editor. It keeps the original's requestor member information.
- In All Groups mode, both buttons first ask *"Which data holder group?"*. An agreement belongs to one data holder group.

Basic Info

- **Template Name** (required), e.g. "Law Enforcement Access".
- **Data Holder Group**: when you create a template, this is the group you are working in, and the template stays in that group. When editing, only a master account can change it.
- **Short Description**: up to 25 characters.

- **Required Group Types**: the intended requestors, e.g. *law-enforcement, government*.
- **Disclosure Mode**: **Open** (shown as-is), **Hashed** or **Omit**.
- **Description**.
- **Visibility**: Private, Public or Globally published, as in the list.
- **Active**: whether the template accepts new subscriptions. New templates start inactive.

Request Types

Add a request type from the presets **Standard**, **Confidential**, **Exigent** or **ICANN RDRS**, then adjust it. A template needs at least one. Each request type has:

- **Type Code**: a globally unique hash the system assigns on save. It cannot be edited.
- **Name** (required, unique within the template). You cannot save duplicate names.
- **Kind**: **RDAP** or **ICANN RDRS**.
- **Access Level**: L0 - Public, L1 - Basic, L2 - Enhanced or L3 - Full.
- **Description**.
- **Conf**, **Exig**, **Manual Appr.** and **Active** checkboxes. Inactive request types are shown faded.
- **RDAP Params**: the fields disclosed, grouped as Domain, Nameservers, Events, Registrant, Admin, Tech, Billing, Registrar, DNSSEC, Network, ASN and Other, with **All On** / **All Off**.
- **Query Value Regex** and **Rejection Message**: limit what can be queried under this request type, and set the message returned when a query does not match. Leave the regex blank to allow everything.
- **Custom Parameters**: see below.

An **ICANN RDRS** request type sends a registration data request to ICANN's RDRS using the requestor's own ICANN credentials, so it never reaches a data holder. It has no access level, RDAP parameters, Conf/Exig flags or query regex. Instead, it has **RDRS Defaults**: a **Preset Category**, **Preset Priority** and **Preset Organization**, and **Selectable Data Elements**. If you tick no elements, all of them can be selected.

Custom parameters

Custom parameters let requestors supply extra information, such as case numbers or reference documents. Each has:

- **Parameter Name**: letters, numbers and underscores only.
- **Data Type**: String, Integer, Float, Boolean, Date, DateTime, File, Email, URL, Enum (Select), Text (Multiline) or JSON.
- **Required**, **Description**, **Placeholder** and **Default Value**.

Some data types add their own validation:

- **Enum**: **Enum Values**, a comma-separated list of allowed values.

- **String, Text, Email and URL**: **Max Length** and **Validation Regex**.
- **Integer and Float**: **Min Value** and **Max Value**.
- **File**: **Allowed File Types**, **Max Size (MB)**, and **Format Requirements**, which are checked automatically when the data holder receives the file:
  - **Min Pages** and **Max Pages**.
  - **Min Word Count**.
  - **Required Text** and **Disallowed Text**, one regular expression per line.
  - **Min Width (px)**, **Min Height (px)** and **Min DPI**.
  - **Min Embedded Image Width (px)** and **Min Embedded Image Height (px)**.
  - The checkboxes **Must have selectable text (not a scan)**, **Must contain an image/photo** and **Allow scanned documents (OCR)**.

Leave a format field blank to skip that check. You can reorder parameters by dragging them or with the arrows, and copy one with **Duplicate**.

Rate Limits

Max Queries Per Day and **Max Queries Per Month**. Leave either blank for no limit. A subscription takes the limits in force when it is created, shown as **Query limits** in its details. Later edits apply to new subscriptions only.

Terms & Policy

Contact information is shown to requestors with the agreement. **Use the data holder group's contact information** is on by default and shows the contact set on the group (2.7). Turn it off to give this agreement its own **First name**, **Last name**, **Contact email** (required), **Contact phone** (optional), **Street address**, **Country**, **City**, **State / Province** and **Postal code**. Changing the contact never asks subscribers to accept anything again.

Data Holder Group Fields are the legal terms your group sets:

1. Click **Add section** and enter a section title, or choose from **Insert standard section...** (Purpose, Requestor Group Membership, Permitted Uses of Disclosed Data, Restrictions and Prohibited Conduct, Recourse and Enforcement, Payment and Cost Recovery, Transparency, Handling, Processing, Testing, Authentication Service Quality Standards, Requestor Group specific Provisions, and Term and Termination, Governing Law and Legal Venue). A standard section comes with its clauses already written.
2. **Add clause** adds a clause to a section. Sections and clauses are numbered automatically (1, 1.1, 1.2 ...), and the arrows reorder them.
3. **Insert reference...** adds a cross-reference to another section or clause. In the editor it appears as a chip showing the number and section name. In the agreement it becomes the

section number, which stays correct when sections move. A chip whose target was removed shows *"Reference no longer exists"*.

Requestors see each section with its own acceptance checkbox. Untitled sections are not saved.

Requestor Group Fields are the values the requestor group must supply when subscribing. Click **Add field** and set:

- **Name** (required and unique).
- **Type**: Text, Paragraph, URL (link to review), Number, Date, Select or Checkbox.
- **Required**.
- **Help text**.
- **Options** for Select fields.
- **Character limit** for Text and Paragraph fields.
- For a URL field, the **Link URL** of a document the requestor opens and reviews. With **Acknowledgement required** ticked, the requestor must confirm they have read it before subscribing.

Requestor Member Information is the information every member of the requestor group must have on their profile to make requests under this agreement. Members missing any of it cannot use the agreement.

- Tick the standard fields you need: **First Name**, **Last Name**, **Email**, **Phone Number**, **Street Address**, **City**, **State / Province**, **Postal Code** and **Country**.
- **Add Custom Field** adds your own field with a **Field name** and **Help text**. The trash button removes it. A custom field cannot reuse a standard field's name, and each field may appear only once. A custom field with no name is not saved.

Publishing changes to subscribers

Editing a template does not change active subscriptions, because each one keeps its pinned copy of the terms. To pass changes on, click **Publish changes to subscribers**.

1. Each active subscriber is listed with its state: *Awaiting response*, *Older copy* or *Up to date*. A subscriber has an older copy when its pinned legal sections, requestor group fields, requestor member information or request types differ from the template's. Contact, rate limit and visibility changes do not count.
2. Choose **Force** or **Propose** for each subscriber, or use **Force all** or **Propose to all**. Propose is the default.
3. For forced changes, set how many days the requestor has to accept them (30 by default).
4. Click **Apply**.

A proposed change is optional: the subscription stays as it is unless the requestor accepts. A forced change must be accepted by the deadline, or the subscription is terminated and becomes CANCELLED.

2.4 Data Holders

ABBREVIATIONS

RDAP	Registration Data Access Protocol

CONCEPTS

Data holder	An operator of an RDAP server holding registration data, running Jareg. It can belong to several data holder groups, and every active data holder in a group is a candidate to answer that group's subscription tests. Disabling one takes it out of those tests; deleting one deactivates its credentials, which cuts its connection at once.
Credentials	The Client ID and Client Secret this group admin issues for one data holder, entered in Jareg under Settings → Group Admin (3.8). Jareg uses them to read the agreements of its groups and the subscriptions it must honour, including each subscription's introspection details. The secret is shown only once and cannot be recovered, only replaced.
URL	The address of the data holder's Jareg. This group admin calls it to run the retrieval part of each subscription test, so a data holder without a reachable URL cannot answer tests.

1. **Register Data Holder**: enter **Data Holder ID** (required, and fixed once registered), **Name** (required), **Data Holder Groups** (at least one), **URL**, **Contact Email**, **Callback URL** and **Description**.
2. When you save, the **Credentials Generated** panel shows the **Client ID** and **Secret**. Copy them then, because the secret is shown only once. The data holder enters them under Settings → Group Admin in Jareg (3.8).
3. **View Credentials** shows the Client ID later, but the secret cannot be recovered. **Regenerate** issues a new secret and invalidates the current one, so the data holder must update its settings.

The list is scoped by the header's group selector and can be searched by name, ID, URL or contact. Click the status badge to enable or disable a data holder. Deleting a data holder deactivates all its credentials, and any active connections stop working immediately.

> **Limit:** only master accounts can register, edit, enable, disable or delete a data holder, or regenerate its credentials. Admin and User accounts add data holders to their groups by approving applications (2.5).

2.5 Applications

"Review applications from organizations wanting to join a data holder group."

ABBREVIATIONS

RDAP	Registration Data Access Protocol
TLD	Top-Level Domain

CONCEPTS

Application	A request from an organization to join one data holder group as a data holder. Approving it registers the data holder in that group and issues its credentials; a denied application creates nothing. Only one pending application is accepted for each contact email and group.
Private group	Private groups are left out of the joining list, so an applicant must type the group's name (capitals do not matter). A disabled group accepts no applications.

1. A prospective data holder opens the public **Join a Data Holder Group** page. They choose the data holder group from the list, or pick **Know a private group name? Enter it manually** and type the exact name. They then complete **Organization Information**, **Organization Admin Contact**, at least one RDAP server URL, at least one supported TLD (including the leading dot) and **Additional Notes**. When they submit, they receive an Application ID.
2. In **Applications**, filter by All, Pending, Approved or Denied, search by organization, contact or TLD, and click **View Details** to review the organization, address, admin contact, RDAP servers, supported TLDs and notes.
3. **Approve** creates the data holder and generates credentials, which are shown once in the confirmation dialog. **Deny** records the decision. **Review Notes (optional)** are recorded with either decision along with the reviewer and date. For a denial, the notes hold the reason.

You can also delete an application. Deletion cannot be undone.

2.6 Requestor Groups

ABBREVIATIONS

DH, DHs	Data Holder, Data Holders
API	Application Programming Interface

CONCEPTS

Requestor group	The organization requestors belong to, such as an agency, identified by a group code that must be unique here. It registers from Jadmin and is assigned to one data holder group, whose administrators review it.

Requestor groups register with this group admin from Jadmin. This page is where you review them.

- Filter by **All**, **Pending**, **Accepted**, **Denied** or **Suspended**, and search by name, code or contact.
- The table shows Code, Name (with organization), DH Group (in All Groups mode), Type, Contact and Status. For an accepted group, the status badge reads Active or Disabled, and clicking it toggles the group.
- **View Details** shows the code, type, organization, contact, description, reason for access, status, registration date, and who reviewed the group. You can reassign the group from the **Data Holder Group** dropdown here.
- **Accept** generates API credentials. The button reads **Accept & Generate Credentials**, and the new Client ID and Client Secret are shown once. **Deny** can record a reason in **Notes**.
- **Credentials** (on accepted groups) shows the Client ID. The secret cannot be recovered, and **Regenerate** replaces the pair. The requestor group enters these credentials in Jadmin to use this group admin's API.
- Deleting a requestor group deactivates all its credentials, and active connections stop working.

2.7 Data Holder Groups *(master accounts only)*

CONCEPTS

Data holder group	A set of data holders that offer agreements together. Agreements, data holders, requestor group registrations and accounts are all attached to groups, and an account that is not a master only sees and acts on the groups it belongs to.
Visibility	The group's setting is the ceiling for its templates. **Private** hides the group from the joining list and lists none of its templates, **Public** shows the group and lets its templates be listed, and **Globally published** also lets its Globally published templates be published beyond this group admin. Disabling a group makes all its templates private in effect and closes it to applications.
Administrator contact	Shown to requestors with every agreement in the group unless the agreement has its own contact. Changing it never asks subscribers to accept anything again.
Group members	Admin and User accounts work only within the groups they are members of. Master accounts reach every group without membership.

1. Click **Create Group** and enter:
   - **Name** (required) and **Description**.
   - **Visibility**. **Private** hides the group from the joining list and lists none of its templates anywhere; applicants must enter the exact group name. **Public** shows the group in the joining list and lists its templates. **Globally published** does the same and also lets the

group's Globally published templates be published beyond this group admin. The group's visibility is the ceiling for its templates.

  - **Administrator contact**: **First name**, **Last name**, **Contact email** (checked for a valid format), **Contact phone** (optional), **Street address**, **Country**, **City**, **State / Province** and **Postal code**. This is how requestors reach the group about its agreements, and every agreement shows it unless the agreement has its own contact.

2. Click the status badge in the list to enable or disable a group.
3. **Manage Members** assigns users to the group. Master accounts have access to all groups automatically and need no membership. Removing a member removes their access.
4. A group can only be deleted once no data holders are assigned to it. Deleting it removes all member associations.

2.8 Instances *(master accounts only)*

"Data Holder Instances"

ABBREVIATIONS

SSL	Secure Sockets Layer (TLS certificate)

CONCEPTS

Instance	A Jareg deployment that this group admin hosts for a data holder, with its own database, containers and subdomain. Creating an instance does not register it as a data holder here: register it on Data Holders (2.4) and enter the credentials in its Settings → Group Admin (3.8), as for any other data holder.
Stop, Update and Teardown	**Stop** halts the instance's containers and keeps its database, so **Start** brings it back as it was. **Update** replaces the containers with the latest images and also keeps the database. **Teardown** deletes the containers, the database and the web server configuration for good.

Optional hosted deployments of the Data Holder application.

- **New Instance**: enter **Data Holder Name**, **Subdomain** (3–30 lowercase letters, numbers and hyphens; the resulting URL is shown), one or more **Data Holder Groups**, and optionally a **Data Holder ID** and **Admin Password** (both are generated when left blank). **Create Instance** creates the database and containers, configures Nginx, and obtains an SSL certificate. This takes about 30–60 seconds.
- Each instance card has **Stop** / **Start**, **Update** to the latest image, and **Teardown**. The **Details** button shows the URL, groups, database, ports, containers, images, admin username and password (**Show** / **Hide**), and live container status.
- **Update All** restarts every running instance with the latest images.

Teardown permanently stops and removes the containers, drops the database and all its data, and removes the Nginx config. It cannot be undone.

2.9 Users *(master and admin accounts)*

ABBREVIATIONS

DH, DHs	Data Holder, Data Holders

CONCEPTS

Account type	**Master** accounts reach every data holder group, and only they can use DH Groups, Instances, Settings and Audit Logs or register data holders. **Admin** accounts work in their own groups and manage the accounts there. **User** accounts work in their own groups without access to this page.
Account hierarchy	You can only create or edit accounts of a lower type than your own, and you cannot promote anyone to your own type or above, so Master accounts cannot be created here. Admins can assign only the groups they belong to, and nobody can change their own group access or status.

- **Add User**: enter **First Name**, **Last Name** and **Email** (all required), **Password** (when editing, leave it blank to keep the current one), **Type** (Master, Admin or User), and **Data Holder Groups**. Admin and User accounts must belong to at least one group. Master accounts have access to all groups automatically.
- You can only create accounts below your own access level, and admins can only assign groups they belong to. Memberships a user has in groups outside your access are not changed.
- Only the account owner or a master account can change an email address.
- You cannot delete or deactivate your own account. Deleting a user also removes all their group memberships.

2.10 Audit Logs *(master accounts only)*

ABBREVIATIONS

CRUD	Create, Read, Update, Delete
API	Application Programming Interface

CONCEPTS

Category	**AUTH** covers sign-ins and password changes, **CRUD** creating, changing and deleting records, **WORKFLOW** subscription review, testing and activation, **CREDENTIAL** issuing, viewing and regenerating credentials and registering signing keys, and **API** calls from Jadmin, data holders and the public registration and joining forms.
Result	**FAILURE** marks failed sign-ins and password changes and refused subscription submissions. Calls refused for missing or wrong credentials are recorded under **API** with the action AUTH_FAILED or ACCESS_DENIED; filter by **Action** to find them.
Performed By	Many record changes made in this console are recorded as "admin" rather than by name, while sign-ins and test runs carry the email of the account that made them.

- The header shows the total events logged, events in the last 24 hours, and failures.
- Filters: **Category** (AUTH, CRUD, WORKFLOW, CREDENTIAL, API, SYSTEM), **Entity Type** (USER, TEMPLATE, SUBSCRIPTION, DATA_HOLDER, REQUESTOR_GROUP, CREDENTIAL, REQUEST_TYPE, SYSTEM), **Action**, **Performed By** and **Result** (SUCCESS, FAILURE, DENIED), plus a search box covering entity name, ID and details. **Clear Filters** resets them.
- The table shows Time ("just now", "5m ago", "3h ago", "yesterday"), Category, Action, Entity, Performed By and Result.
- Opening an entry shows **Audit Log #{id}**, with the full timestamp, source, IP address, entity type, ID and name, and the recorded details.

2.11 Settings *(master accounts only)*

ABBREVIATIONS

API	Application Programming Interface

"Deployment-wide settings for this Group Admin instance." The **Logo** tab holds the logo and login notices; the **Common Repository** tab publishes this deployment to a common repository, where requestor managers can find it.

Logo

- **Upload logo** replaces the default logo on the login screen and in the header. It accepts PNG, JPEG, GIF or WebP, up to 2 MB. A wide image with a transparent background works best.
- The current logo is shown with its file name, size, who uploaded it and when, labelled *Your logo* or *Default logo*.
- **Replace logo** swaps it, and **Remove** goes back to the default.

Login notices

- This is extra wording shown under the sign-in form, such as a privacy notice or a link to your terms. Nothing appears until you add an entry.
- Each notice has **Text** and an optional **Link (optional)**. Add a link to turn the text into a link, or leave it blank for plain text.
- **Add notice** creates an entry, **Move up** / **Move down** set the order, and **Remove** deletes one. You can add up to 10 entries.
- **Save notices** saves the list.

Common Repository

Publish this registry to a common repository: a common repository is the central node a Jadmin consults when it looks for data holder groups (1.6). The announcement describes this deployment as a whole and lists the data holder groups set to Globally published, each with its templates that are also Globally published (2.3, 2.7). Nothing else is sent, and the repository's own administrators review the listing before it is published.

- A badge shows where the listing stands: *Not registered*, *Awaiting review*, *Listed*, *Rejected* or *Suspended*.
- **Announce this registry to a common repository** turns publication on. While it is off nothing is sent, and this deployment stays invisible to anyone who has not entered it by hand.
- **Common repository URL**: the repository's base address, the part before */registry*. Jaremin's server calls it, not your browser, so it must be an address the server can reach.
- **How this registry is identified over there**:
  - **Registry code**: claimed on the first announcement and fixed afterwards. Requestor managers identify the deployment by it.
  - **Registry name**: the name of this deployment, not of any one data holder group.
  - **Description**.
  - **Public API base URL**: where requestor managers should send their requests, as the outside world reaches this deployment.
  - **Contact email**.
- **Save** stores the settings. **Announce now** sends the announcement at once (turn publication on and save first); otherwise it is sent every 15 minutes. The result of the last announcement is shown beneath, with any error.
- **What would be published ({groups} groups · {templates} templates)** previews the listing: each group with its templates, their request types, manual approval, parameters and query limits. A Globally published template under a group that is not itself Globally published is not listed.

3. Jareg — Data Holder

Navigation: **Dashboard**, **Pending Requests** and **Data Holder Groups**; a **Data** menu (**RDAP Data**, **RDAP Query**); **Policy** and **Logs**; and a **Settings** menu (**Group Admin**, **External Database**, **File Automation**, and **Users** for master accounts). The header shows the signed-in user, with a shield icon for master accounts, and **Logout**.

> **Note:** an idle session ends after a set time (15 minutes by default). Before it ends, a *"Still there?"* dialog counts down and offers **Stay signed in** or **Sign out**. Once the session has expired, the sign-in page reports *"Your session has expired. Please sign in again."*

3.1 Dashboard

"Overview of your RDAP Data Holder status and activity"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
L0–L3	Access levels 0 (Public) to 3 (Full)

The page refreshes itself every 5 seconds (a green dot and a *Last updated* time show this), and **Refresh** reloads it at once.

- Counters: **Pending Requests**, **Approved (24h)**, **Denied (24h)** and **Test Domains**.
- **Requests Over Time** charts RDAP queries per day. Choose **7d**, **30d** (the default) or **90d**, or **Custom** to enter **From** and **To** dates and click **Apply**.
- **Request Outcomes** is a bar of Approved, Pending, Denied and Cancelled requests, each named with its count.
- **Recent Pending Requests** lists the five newest, with Query, User, Level and Created. **View All** opens Pending Requests.
- **Recent Activity** lists the five newest log entries, with Query, Result, Level and Time. **View All** opens Logs.
- **Available Test Domains** lists the domain names held in your local data. **Test Query** opens RDAP Query.
- **JAKE Compliance**: click **Load** (then **Refresh**) to fetch what your Group Admin connections report. Each data holder group appears with an *Active* or *Inactive* badge and its description. Under it are its active templates, each with a **Max: L**n badge for its highest access level and *Confidential* / *Exigent* markers where a request type allows them. Each template lists its **Request Types** with their access level and a lock (confidential), lightning (exigent) or *Manual*

marker. **Active Subscriptions** lists the subscriptions in force with their template and access level. If nothing comes back, the page says to check that the Group Admin connection is configured.

3.2 Pending Requests

"RDAP Requests — View and manage all data access requests"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
SHA-256	Secure Hash Algorithm, 256-bit
PDF	Portable Document Format

CONCEPTS

Request record	Every RDAP query this data holder answers is recorded here, not only the ones that need review. Queries answered at once appear as *APPROVED* or *DENIED* with **SYSTEM** as the reviewer. Denied entries include lookups that failed, such as a name that was not found or a request type the requestor group may not use.
Manual approval	A request is held as *PENDING* when its request type requires manual approval or when **Require Manual Review for All Requests** is on (3.8). A request type with no manual approval setting counts as requiring it. The requestor gets a request ID to poll with, and you decide here.
Access level	How much a request may see, from Level 0 Public to Level 3 Full, taken from the requestor group's subscription or from the request type and shown in the **Access Level** column. Approving grants exactly that level: you cannot raise or lower it. A query made under no agreement gets Level 0 by default.
Expiry and polling	A held request expires 24 hours after it was made: it leaves the pending count, can no longer be approved or denied, and an approved answer can be collected by the requestor only within that time. A denied request returns your **Denial Reason** when the requestor polls. The requestor can cancel a request while it is pending, which shows here as *CANCELLED*.
Confidential and Exigent flags	*Exigent* marks an urgent request: if its request type allows exigent requests it skips manual review, even when **Require Manual Review for All Requests** is on, and otherwise the flag is ignored. *Confidential* asks that the registrant not be told about the request. It is recorded with the request.
Requestor member fields	An agreement can require details about the individual requestor, which reach the data holder in the requestor's token. If a required field is missing, the query is refused before any lookup. The values supplied appear under **Member Information**.
File scanning	Jaddar sends attached files to the data holder before the query, and each is checked for an allowed type, a size of at most 25 MB, contents that match the extension, and no scripts or unsafe names. A file that fails is rejected and logged as a security threat (3.7), so stored files show *Clean*. When a document scanner is configured, files are also checked against the file rules the request type sets.

The list refreshes every 5 seconds; **Refresh** reloads it at once.

- Tabs with counts: **All**, **Pending**, **Approved**, **Denied** and **Cancelled**. The search box covers the query and the requestor.
- Columns: Request ID, Query (value and type), Requestor (username and email), Files (count and file types), Agreements (the requestor group code and request type code, or *None* for a public query), Access Level, Status, Created, and Reviewed (time, and the reviewer's name).
- Row actions: **Review** (pending requests only), the eye button for **View Details**, and delete (with confirmation).

Reviewing a request

1. Click **Review**. The **Review Request** window shows the request details (below).
2. To deny, enter a **Denial Reason**. It is required only for a denial and is returned to the requestor when they poll for the request status.
3. Optionally add **Admin Notes (Optional)**, recorded with the decision.
4. Click **Approve** or **Deny**, or **Cancel** to leave the request pending.

Request details

View Details (and the review window) shows the Request ID, Status, Query Type, Query Value, Requestor, Email, **Groups** and **Agreements**, then where present:

- **Admin Notes** from the review.
- **Member Information**: the requestor member fields the agreement requires, as supplied.
- **Custom Parameters**: the values the requestor supplied. File parameters are listed under **Files (**n**)** with type, parameter name and size. **Examine** opens a preview (PDF, JPEG, PNG and text can be previewed; other types must be downloaded) with a **Download** button. **Download All** appears when there are several files. A file whose content never reached the data holder is marked *Reference only*.
- **Attached Files**: each file with its scan status (*Clean*, *Malicious*, *Suspicious*, *Pending Scan* or *Scan Error*). A malicious file is shown as *Blocked* and cannot be opened. Others have **Examine**, which shows the file's SHA-256 and a preview.
- For a denied request, **Denial Status** with the reason (or *"No reason was provided."*). For a reviewed request, **Reviewed At** and **Reviewed By**. For an approved request, the **Response Data** returned.

3.3 Data Holder Groups

"Groups this data holder belongs to, their members, and the agreement templates available to each group."

ABBREVIATIONS

RDAP	Registration Data Access Protocol

CONCEPTS

Data holder group	A set of data holders that offer agreements together, administered in Jaremin. Your data holder is a member of each group listed here, and the group's agreements decide who may query you and at what level.
Live Group Admin data	Jareg keeps no local copy of groups, agreements or subscriptions. This page, the JAKE data and every query under an agreement read them live from all active Group Admin connections, merged together, and a connection that fails is skipped.
Agreement and request type	An agreement (shown as a template) holds one or more request types, each setting an access level, the RDAP fields disclosed, any custom parameters, an optional pattern the query must match, and whether confidential and exigent requests are allowed. *Manual* means requests of that type wait for your review in Pending Requests, and *Auto* means they are answered at once. A request type with no setting counts as *Manual*.
Subscription	A requestor group's acceptance of one agreement, reviewed and activated in Jaremin. A query under an agreement names the requestor group code and the request type code. It is accepted only if the Group Admin reports a subscription for that group that offers that request type.
Token introspection	For each query under an agreement, the data holder asks the requestor side to confirm the requestor's bearer token, using the introspection URL and client credentials recorded on the subscription, which reach it through the Group Admin. The token's groups or roles must include the requestor group's code or name. The check runs on every query, with no caching.
JAKE compliance	JAKE data describes what this data holder offers: its groups, their active templates with request types, access levels and flags, each template's agreement code, and the active subscriptions. It is read live from the Group Admin, shown on the Dashboard, and added to an RDAP answer when the requestor asks for it. A template's disclosure setting in Jaremin can show its details as they are, hash them, or leave the template out.

This page is read-only. It gathers what every active Group Admin connection reports; groups, templates and subscriptions are managed in Jaremin. **Refresh** reloads it. Without a connection the page shows **Group Admin Not Connected** with a link to **Settings**.

- **Group Membership** tab: one row per group with Group, Group Admin (the connection it came from), Members, Templates and Status. Expand a row to see:
  - **Members**: Name, ID, Contact, URL and Status, with your own entry marked *You*.
  - **Agreement Templates**: Template (with short description), Template ID, Request Types and Status. Each request type shows its access level and a *Manual* or *Auto* badge. Hover its info icon for the type code, access level, confidential, exigent and manual approval settings, query regex and custom parameters.

- **Requestor Subscriptions** tab: the same groups with a Subscriptions count. Expand a row to see each requestor group (with its group code), its template and the subscription status. Hover the info icon by a requestor group for its type, organization, contact, email, phone, address, agent ID and purpose.

3.4 RDAP Data

"RDAP Data Management — Manage domains, IP networks, and ASNs"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
ASN	Autonomous System Number
AS	Autonomous System (a network identified by an ASN)
DNS	Domain Name System
DNSSEC	Domain Name System Security Extensions
DS	Delegation Signer (DNSSEC record)
JSON	JavaScript Object Notation
CSV	Comma-Separated Values

CONCEPTS

Test data	Records marked with **Mark as Test Data** are the ones a subscription test uses: Jaremin asks this data holder to check every marked domain, IP network and ASN against the agreement under test. With no marked records, this data holder skips the test and Jaremin tries another member of the group. Test records are not hidden: real queries are answered from them like any other record.
Local and mapped data	A domain lookup first tries each active external database mapping (3.9) and then this page's local records. IP network and ASN lookups always use the local records. The **Mapped Data** tab only displays mapped domains; edit them in their own database.
Policy link	A record can be tied to one policy through the **Policy expression ID** import field. A linked record always uses that policy. Otherwise the policy is chosen by scope conditions or the default (3.6).

- Counters: **Total Entities**, **Domains**, **IP Networks** and **ASNs**.
- Tabs: **Domains** (Domain, Handle, Status, DNSSEC), **IP Networks** (Handle, Start Address, End Address, Version, Network Name) and **ASNs** (Handle, Start AS, End AS, Name, Country). A **Mapped Data** tab appears when an external database mapping is active (3.9), with the number of sources.
- Toolbar: search, **CSV** (import), **JSON** (import), **Export** (writes the current tab's records as JSON), refresh, and **Add Domain** / **Add IP network** / **Add ASN**.

- Records marked as test data show a *TEST* badge. Tick rows to **Delete Selected** or **Clear Selection**. Row buttons edit or delete one record. Pages show 25, 50, 100 or 200 records.

Adding or editing a record

The editor has collapsible sections: **Basic Information**, **Events**, **Nameservers** (domains), **Links**, **Remarks & Notices**, **Secure DNS (DS Records)** (domains) and **Contacts (Entities)**. Each contact has a handle, name, organization, email, phone, fax, roles and address. Under Basic Information, tick **Mark as Test Data** for records that subscription tests should use.

Importing

- **CSV** (*"Import … from CSV"*): drop or choose a file, set the **Delimiter** (Comma, Semicolon, Tab or Pipe) and **First Row is Header**, then click **Preview & Map Fields**. Map each column to an RDAP field or **-- Skip --**. **Auto-Map** matches on header names, and **Expand All**, **Collapse All** and **Clear All** manage the mapping list. Fields holding personal data carry a *Sensitive* badge. Click **Import** n **Fields**. The result reports Total, Imported and Failed, with **View** n **Errors** listing each failed row.
- **JSON** (*"Import … from JSON"*): upload a file or paste a JSON array. **Load Template** inserts an example and **Format** tidies the text. Nested contacts with roles are supported. Click **Import JSON**.

Mapped Data

A read-only view of the domains served through active mappings. With more than one mapping active, choose one from the list, labelled *Local* or *External*. Search, then page through Domain Name, Handle, Zone, Registration, Expiration, Last Changed, Nameservers and Contacts.

3.5 RDAP Query

"Check that your data can be retrieved and read correctly"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
ASN	Autonomous System Number
AS	Autonomous System (a network identified by an ASN)
JSON	JavaScript Object Notation

1. Choose **Domain**, **IP Address** or **ASN**.
2. Enter the **Domain Name**, **IP Address** or **AS Number**. For domains, **Quick Select** offers up to eight of your domain names.
3. Click **Execute Query**.

The query runs with your data holder credentials at Level 3, so with the standard redaction matrix the results show your full, unredacted data. It is never held for review. **Query Results** shows *Success* or *Failed* and the response as JSON.

3.6 Policy

"Policy Expressions & Redaction — Configure access control policy expressions and data redaction rules"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
AS	Autonomous System (a network identified by an ASN)
DNS	Domain Name System
V0–V3	Validation levels 0 (None) to 3 (Identification)
JSON	JavaScript Object Notation

CONCEPTS

Policy expression	A policy says how sensitive each field is for the records it covers, and one policy is chosen per answer: a policy linked to the record, otherwise the first active, non-default policy whose scope conditions all hold, otherwise the active default policy. When several policies match, none takes precedence, so keep their scopes from overlapping. With no policy at all, every field counts as Level 0 and the standard matrix shows it in full.
Scope conditions	Conditions are checked against the record's name, handle, object class, status and WHOIS server, the registrant's organization, name, email and phone (through the **Organization** field or a custom path), and values from custom tables and contact custom fields.
Default policy	The fallback when no linked or matching policy applies. It must be active to be used, and only one policy should be default: **Set as default** clears the others. Imported policies have no scope conditions, so they take effect only as the default or when linked to records.
Sensitivity level	Each field's rating from Level 0 Public to Level 3 Full, and the only per-field input to redaction. Fields a policy does not list count as Level 0. When several entries land on the same response field (street, city and postal code all belong to the address, for example), the highest level wins.
Validation level	How thoroughly a field's value has been checked, from V0 None to V3 Identification. It is reported to the requestor with the sensitivity level (Jaddar shows both as markers on each field), but it does not affect redaction.
Redaction matrix	One matrix shared by all policies, saying for each access level, sensitivity level and value state whether to return the value, the word "REDACTED", or nothing. The standard matrix leaves out empty values, shows a value when the access level is at least the field's sensitivity, and redacts it otherwise. A missing row returns the full value.

Custom role	Contact groups in an import file must match a built-in role (by keyword, so any name containing "admin" counts as Admin) or an active custom role, or the whole file is blocked. That is why a custom role name may not contain a built-in keyword.

Three tabs (**Policy Expressions**, **Redaction Rules** and **Custom Roles**), with **Import JSON** and refresh buttons above them.

How the pieces fit together

A **policy expression** decides which rules apply to a given RDAP object, using scope conditions. Each policy carries **field sensitivity levels**: a sensitivity rating, and optionally a validation level, per RDAP data element. At query time the **redaction matrix** combines a field's sensitivity with the requestor's access level, and whether the value is empty, to decide whether the value is returned in full, replaced with "REDACTED", or left out.

> **Recommended order:** define custom roles, import or create policies, review field sensitivity levels, then tune the redaction matrix.

Custom Roles (do this first)

"Define contact roles beyond the built-in set so policy imports can map their groups to distinct fields."

- **Built-in Roles**: Registrant, Admin, Tech, Billing and Abuse. These always exist and cannot be edited.
- **Add Role**: enter a **Role Name** that matches the contact group name used in the import file (case-insensitive) and an optional description. A field key (for example *accountHolder*) is derived automatically. A custom role may not duplicate or shadow a built-in one.
- The list shows Name, Field Key, Description and Status, with edit, **Disable** / **Enable** and delete. Deleting asks for confirmation: existing imported policies keep their fields, but future imports that reference the role are blocked.

> **Why first:** if an import file references a contact group that is not a built-in role (for example "Account Holder"), the import is blocked with *"Import blocked — undefined contact role(s): …"* until you define a matching custom role.

Importing policy JSON

1. Click **Import JSON** and drop one or more *.json* files on the drop zone, or click to browse.
2. The files are uploaded and parsed. The preview lists one card per file with the policy name and file name. Expand a card to read its description. Files that fail parsing are listed separately with the reason and are left out. **Remove** drops a file from the batch.
3. Click **Import** n **Policy Expressions**. The results list each imported policy by name and each failure by file with its error.

What the importer does with a file

- Requires a top-level **rules** array. Each rule carries its element name, group and category, with **sensitivity** and **validation** as levels from 0 to 3. Rules whose **collect** value is "Omit" are skipped.
- Contact-group rules must resolve to a defined role, built-in or custom. Structural groups (DNS records, registration scope, op status, payment and transaction) need no role.
- Rules for the same object type and field are merged, keeping the higher sensitivity and validation level.
- The policy name comes from the file's **description**. A name already in use fails that file. The policy's description collects the file's notes, file name, **org_name**, a summary of **scope_attributes**, **effective_date** and **version**. The scope attributes are only described: no scope conditions are created.
- Imported policies arrive active and not default.

Creating or editing a policy by hand

Click **New Policy Expression** (or **Create First Policy**), or click a policy card to edit it.

- **Basic Information**: **Policy Name** (required) and a description of when it applies.
- **Scope Conditions**: rows reading *When {field} {operator} {value}*, all of which must hold. Pick a field grouped as Domain, Identity, Status, Scope, Registrant, Admin, Tech, Billing, Contact, Location, Network or Other, or enter a custom dot-notation **Field Path** (for example *entities.0.vcardArray*). Operators: equals, does not equal, contains, does not contain, starts with, ends with, is empty, is not empty, is one of, is not one of, and matches regex. **Add Condition**, **Duplicate** and **Remove** manage the rows. A policy with no conditions is never chosen by scope: it applies only as the default or when linked to records.
- **Note to Requestor**: optional text returned as a notice in RDAP responses when not empty.
- **Policy Status**: **Active** (only active policies are evaluated) and **Default Policy** (used when no other policy matches). Prefer **Set as default** on the card, which also clears the previous default.
- **Field Sensitivity Levels**: see below.

The list has a search box and an **All Policies** / **Active Only** / **Inactive Only** filter. Each card shows its scope conditions, defined fields, number of redaction rules, note to requestor, and created and updated dates, with **Set as default**, **Activate** / **Deactivate**, edit and delete.

Field sensitivity levels

Inside a policy, use **Add Field** or **Quick Add Common Fields**. Each entry has an **Object Type** (Domain, Entity, Nameserver, IP Network, AS Number or All Objects), a preset or custom **Field Path** in dot notation (for example *vcardArray.email*) with a **Display Name**, a **Sensitivity Level** (Level 0 Public, Level 1 Basic, Level 2 Enhanced, Level 3 Full), an optional **Validation Level** (V0 None, V1

Syntactical, V2 Operational, V3 Identification), and **Rule is enabled**. The list shows Field, Sensitivity and Validation per object type, and **Filter fields…** narrows a long list.

Redaction Rules matrix

"Redaction Rules Matrix": 32 rows, one for each access level, sensitivity level and value state (*Has Value* or *Empty*). Each row's **Behavior** is chosen from a list:

- **Full Value**: return the actual data value.
- **REDACTED**: return the literal string "REDACTED" in place of the value.
- **Nothing**: leave the field out of the response entirely.

Filters for **Access Level**, **Sensitivity** and **Value** narrow the view, and a **Behavior Reference** legend sits below the table. On a fresh installation with no rules, **Initialize Defaults** fills in the standard matrix.

3.7 Logs

"Audit Logs — Unified activity log — RDAP queries, logins, file uploads, security threats, and system events"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
MIME	Multipurpose Internet Mail Extensions (file content type)
SHA-256	Secure Hash Algorithm, 256-bit

- **Filter**: **All**, **RDAP Queries**, **Security Threats**, **File Uploads**, **Logins** and **Failed Logins**. **Severity**: **HIGH** or **CRITICAL** (click again to clear).
- The **Activity Log** table shows ID, Event, Query / Details, Requestor (or *Anonymous*), Result (SUCCESS, DENIED or PENDING), Access Level, Response Time and Timestamp, with a severity badge where one applies.
- Expand an entry for the event type, query type and value, IP address, user agent, agreements and result message. File and threat events add **Threat Details**: filename, file size, MIME type and SHA-256.
- Event types include RDAP Query, Login, Logout, Login Failed, File Upload, Security Threat, Rule Created / Updated / Deleted, Request Reviewed, Config Changed, and User Created / Updated / Deleted. Security threats are classed as Malicious File, Disallowed Type, MIME Mismatch, Oversized File, Embedded Script, Path Traversal, Polyglot File or Suspicious Content.

3.8 Settings (Group Admin)

"Settings — Configure the Jareg application settings"

ABBREVIATIONS

RDAP	Registration Data Access Protocol

CONCEPTS

Credentials	A Group Admin connection is your data holder's link to a Jaremin instance. Its **Client ID** and **Client Secret** are issued for your data holder in Jaremin, entered here, and sent with every call; the secret is never shown again once saved. You can connect to several Group Admins at once: their data is merged, and a disabled connection is ignored.
Without a connection	With no active connection, queries under an agreement are refused because no subscription can be found, the Data Holder Groups page and JAKE data are empty, and only public answers (Level 0) are given.
Require manual review override	When on, every request is held for review, including public queries, whatever its request type says. The one exception is an exigent request whose request type allows exigent requests: it is still answered at once. When off, a request is held only when its request type requires manual approval.
Public answer	A query that names no agreement gets the public record at Level 0 (by default). This applies even when the requestor sends a valid token.
Joining link and Public Information Services	A requestor group can publish a joining link: a page where people apply to join it. With **Offer Requestor Group Joining Links** on, the data holder reads the joining links of requestor groups with active subscriptions (through each subscription's introspection credentials, refreshed every five minutes) and lists them in public answers as a "Public Information Services" notice with one *Join* link per group. Answers under an agreement never carry the notice.

Two tabs: **Group Admin** and **Logo**.

Request Controls

- **Require Manual Review for All Requests**: when on, every incoming RDAP request is held for review before data is released (exigent requests may still bypass it where the request type permits). When off, manual review follows the request type settings.
- **Offer Requestor Group Joining Links**: when on, public answers (queries made under no agreement) also list, under "Public Information Services", where to join each requestor group subscribed to this data holder or its groups that publishes a joining link. When off, public answers carry only the public record.

The card records who last changed these settings and when.

Group Admin Connections

The header shows how many connections are active.

1. Click **+ Add Connection** (or **+ Add Your First Connection**).
2. Enter a **Connection Name** (a friendly label), the **Group Admin Base URL**, the **Client ID** and the **Client Secret** issued for your data holder in Jaremin.
3. Save, then click the **Test Connection** button. The Status column shows *Connected* or *Failed* (hover for the error), or later *OK*, *Error* or *Not tested*, and **Last Connected** shows when the connection last succeeded.
4. Connections can be edited (leave the secret blank to keep the current one), disabled and enabled, or deleted.

Logo

- **Logo**: upload a PNG, JPEG, GIF or WebP image of up to 2 MB to replace the default logo on the sign-in screen and in the header. **Replace logo** and **Remove** (back to the default) follow. The panel shows the file, size, who uploaded it and when.
- **Login notices**: up to 10 entries shown, in order, under the sign-in form (for example a privacy notice). Each has **Text** and an optional **Link (optional)**. **Add notice**, move up or down, **Remove**, then **Save notices**.

Only master accounts can change the logo and login notices. Other accounts see them read-only.

3.9 External Database

ABBREVIATIONS

RDAP	Registration Data Access Protocol
ASN	Autonomous System Number
JDBC	Java Database Connectivity

CONCEPTS

Data mapping	A description of where RDAP data lives in a database with its own layout: which tables hold domains and contacts, which columns hold each field, and how they join. It lets an existing registry database be served without copying it into Jareg. **Same Database** reads other tables in this application's own PostgreSQL database, and **External Database** connects over JDBC.
Lookup order	A domain query tries each active mapping in turn and uses the first that finds the domain, then falls back to the local RDAP Data records. Several mappings may be active, but none takes precedence over another. Only domain lookups use mappings: IP network and ASN tables can be mapped, previewed and browsed, but queries for them always use local records.
Saved database credentials	The username and password are stored encrypted, never sent back to the browser, and kept when you leave the fields blank. Changing the JDBC URL clears them unless you enter new ones. **Test Connection** and **Discover Schema** reuse them only for the exact saved URL, so they cannot be sent to another server.

Allowed databases	JDBC URLs must be PostgreSQL, MySQL or MariaDB, with their standard driver classes. URLs with options that run scripts or load local files are refused. These checks run when you test the connection or discover the schema.
Database access	Jareg only reads through a mapping, but it does not enforce read-only access. Give an external mapping a database account that can only read. **Same Database** uses the application's own account, so the Data Browser can show any of its tables.
Custom table	A table outside the standard RDAP objects, joined to each domain by conditions that must all match. **Data Acting As** Contact turns its rows into contacts on the domain, while other types are returned as additional table data. Custom values can also be used in policy scope conditions (3.6).

Three tabs: **Data Mapping**, **Schema Diagram** and **Data Browser**.

Data Mapping

"Configure how this application reads RDAP data from your database — including external databases with different schemas"

A status line shows which mappings are active, or that the application is using its default database structure. Click **+ New Mapping** (or **Create Mapping**). The editor is a seven-step wizard with **← Back** and **Next →**. **Save Now** is available from any step after the first.

1. **Basic Info**: **Mapping Name** (required) and a description.
2. **Database**: choose **Same Database** (tables in this application's PostgreSQL database) or **External Database** (over JDBC). For an external database supply:
   - **JDBC URL** (required), for example *jdbc:mariadb://localhost:3306/rdap*
   - **Username** and **Password**. Once saved they are not shown again: the fields read *"Saved (leave blank to keep)"*.
   - **JDBC Driver Class (optional)**, for example *org.mariadb.jdbc.Driver*
   - **Schema Filter (optional)**
   - **Column names are case-sensitive**: tick for PostgreSQL, leave unticked for MariaDB or MySQL.

Click **Test Connection**, then **Discover Schema** (**Discover Local Schema** for the same database). The discovered tables are listed with column and row counts.

1. **Tables**: map each RDAP object type to a table, chosen from the discovered schema or typed by hand. Relationships appear as *References* and *Referenced by*, marked as declared foreign keys or inferred from column naming.
2. **Columns**: map each RDAP field to a column. **Auto-Suggest Mappings** fills in the obvious matches. Unmapped fields are left out of responses.
3. **Joins**: **Domain → Contact Joins** (the foreign key column for each role: registrant, administrative, technical and billing, plus the contact table's join key); **Domain → Host**

(Nameserver) Join (the domain column holding nameserver names, the host join column, and a delimiter for multi-value columns); and **Domain Name Composition** (a suffix or zone column and a separator, with a live preview) when the name is split across two columns.

4. **Custom Tables**: **+ Add Table** maps a table that does not correspond to a standard RDAP object: table name, display label, description, an optional **Data Acting As** type, and an optional join (**Enable join**, **Join To**, and one or more **Join Conditions** that must all match, with a query preview). Under **Column Mappings** (or **Auto-Detect Columns**), each column is either **Standard** (mapped to an RDAP field) or **Custom** (a named value with your own alias).
5. **Preview**: save the mapping first, then preview sample data as **Raw Data** and as **Mapped Data** translated to RDAP fields.

New mappings start inactive. Each mapping card shows *Active*, *External* or *Local*, its tables, contact joins and custom tables, with **Edit**, **Activate** / **Deactivate**, and **Delete** (offered only while the mapping is inactive). Activating does not test the connection, so test it first.

Schema Diagram

"Interactive entity-relationship diagram of the discovered RDAP database schema." Choose a **Data mapping** to draw its tables. The page reports an **Architecture Rating** (schema design quality: keys, relationships, indexes) and a **Data Integrity Rating** (how well the database enforces correctness), with table and row counts. Zoom in, zoom out and **Reset view** control the diagram. The **Relationships** legend distinguishes declared foreign keys, inferred (heuristic) links and application-defined ones, and **App-defined relationships** shows or hides the last. **Refresh schema** reads the database structure again.

Data Browser

"Browse and inspect data in mapped databases" Choose a **Database Mapping**, then a table. The table list shows column and row counts and how many columns are mapped. Search across text columns, page through the rows (*"Showing X–Y of Z"*), and use **View full value** or copy on any cell. Empty cells show *NULL*.

3.10 File Automation

"File Automation Rules — Define automated logic for handling documents attached to RDAP requests"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
PDF	Portable Document Format
XLSX, DOCX	Excel and Word files (Office Open XML)

CONCEPTS

Rule priority	Rules are checked in ascending **Priority** (lower numbers first, 100 if not set), with the first enabled rule whose conditions all hold deciding the action and adding to its trigger count. A rule with no conditions matches every file, so give it the highest number.
Conditions	Every condition you fill in must hold, and empty ones are ignored. **Filename Pattern** accepts the wildcards * and ? and ignores case. **Requestor Groups** must match a group in the requestor's token exactly.

A banner explains how rules work and lists the supported file types: PDF, Word (DOCX), Excel (XLSX), Text, JPEG and PNG.

1. Click **New Rule** (or **Create Your First Rule**).
2. **Basic Information**: **Rule Name** (required), description, **Priority (lower = first)** and **Status** (Enabled or Disabled).
3. **Conditions** (leave all empty to match every file): **File Types**, **Max File Size (MB)**, **Filename Pattern** (for example **.report.** or *invoice**_), **RDAP Query Types** (domain, ip, asn) and **Requestor Groups (comma-separated)**.
4. **Action** (**When Conditions Match**):
   - **Auto-Approve**: approve the request automatically.
   - **Auto-Deny**: deny it automatically, with a **Denial Reason** returned to the requestor.
   - **Flag for Review**: send it to manual review with a note attached.
   - **Require Deep Scan**: run a deep security scan before processing.
   - **Quarantine**: quarantine the file and hold the request.

   Any action can carry an **Action Message**.
1. Click **Create Rule**.

The **Automation Rules** list shows Priority, Rule Name, File Types (or *All types*), Conditions, Action and Stats (trigger count), with search, **Enable** / **Disable**, edit and delete.

3.11 Users *(master accounts only)*

"User Management — Manage administrator accounts and access levels"

CONCEPTS

Master	Full access, including this page and the logo and login notices (3.8). Other accounts are sent back to the Dashboard if they open the Users address.
Assistant Administrator	Every other feature, including reviewing requests, policies, Group Admin connections and external database mappings, but not user management or branding.

Your own account	You cannot deactivate or delete the account you are signed in with, so you cannot lock yourself out.

- Counters: **Total Users**, **Master Accounts (page)**, **Assistant Admins (page)** and **Active (page)**.
- **New User**: First Name, Last Name, Username, Email, Password (leave blank when editing to keep the current one) and **Account Type**: **Master** (*"Full access including user management"*) or **Assistant Administrator** (*"All features except user management"*).
- The **Users** table, with search, shows User, Email, Type, Status, Last Login (or *Never*) and Created. Your own row is marked *(you)*.
- Row actions: edit, **Activate** / **Deactivate**, and permanent delete with confirmation. Your own row has edit only.

4. Jaddar — Query Application

The header carries **Home** and **Requests** for every signed-in user, plus **Data Holders**, **Logs**, and **Settings** for administrators only. To the right sit the **RDRS Messages** envelope with its unread count, your name, and **Logout**; administrators also see an **Admin** badge and a **Keycloak** button that opens the Keycloak admin console. A non-administrator who opens an admin page directly is returned to Home without a message.

Sign in with your email address and password; **Forgot password?** sends a reset link, and any login notices an administrator has added appear under the form. On Home and Requests, a *Still there?* dialog counts down two minutes before an idle session ends (15 minutes without activity by default), with **Stay signed in** and **Sign out**. Activity in any open Jaddar tab keeps the session alive. When the session ends you return to the sign-in page with "Your session has expired. Please sign in again."

Release notes are published as a public Atom feed, "Jaddar Release Notes", at */versions.atom*.

Click the **RDRS Messages** envelope to open your Jaddar RDRS mailbox: your assigned RDRS address (marked *not registered* until you confirm your ICANN registration, see 4.1), then the correspondence from ICANN and registrars. Click a message to read it and **Back to messages** to return; **Mark all read** clears the unread count. The count refreshes every minute.

4.1 Dashboard (Home)

ABBREVIATIONS

RDAP	Registration Data Access Protocol
RDRS	Registration Data Request Service (ICANN)
ICANN	Internet Corporation for Assigned Names and Numbers
IANA	Internet Assigned Numbers Authority
TLD	Top-Level Domain
ASN	Autonomous System Number
AS	Autonomous System (a network identified by an ASN)
DNS	Domain Name System
DNSSEC	Domain Name System Security Extensions
L0–L3	Access levels 0 (Public) to 3 (Full)
Conf.	Confidential (request type flag)
Exig.	Exigent (request type flag)
ASCII	American Standard Code for Information Interchange

GDPR	General Data Protection Regulation (EU)
JSON	JavaScript Object Notation
PDF	Portable Document Format

CONCEPTS

Agreement selector	The list shows the agreements your requestor groups hold. Each Keycloak group you belong to is matched by name to a requestor group in Jadmin, which asks every active data holder group for that requestor group's active subscriptions; if a data holder group cannot be reached, its locally cached agreements are shown with a warning. An agreement marked *No credentials* (the Client ID and Client Secret the data holder group issued have not been entered in Jadmin) or *Profile incomplete* (your member profile lacks requestor member fields the agreement requires) cannot be selected.
Request type and access level	Each agreement offers one or more request types, and the request type you pick decides the access level you ask for, from L0 to L3, and which RDAP fields can be disclosed. The data holder answers at that level and then removes anything its policy or the request type's field list excludes, which is why the result reads *Filtered by agreement*.
Public lookup	With no request type selected, Jaddar names no agreement. Servers found through the IANA bootstrap return their public data, and a configured data holder answers as it would any member of the public, at its public level (normally L0). A data holder that offers it adds a *Public Information Services* notice listing requestor groups you can apply to join for fuller access.
JAKE compliance	JAKE is the discovery data a data holder can attach to its answer: its data holder group memberships, the agreements it publishes with their request types, the active subscriptions, and the agreement codes it honours. Turn **JAKE** on and, even on a public lookup, Jaddar records which of your agreements that data holder honours for the queried TLD, so **Filter by data holder response** in the selector can narrow your list to them for the rest of your browser session.
Sensitivity and validation levels	Where the data holder sends them, each field carries S and V badges from its policy: sensitivity says how protected the data element is (0 Public to 3 Full) and validation how thoroughly it was checked (0 None to 3 Identification). A dimmed badge was superseded by another rule that applied to the element; V– means the policy sets no validation level.
Manual verification and denial	A data holder holds a request for review when its settings require review from a set access level upwards or for every request, or when the request type is flagged for manual approval. **Exig.** skips that review only when the request type allows exigent requests; **Conf.** asks the data holder not to notify the registrant, on request types that allow confidential requests. A reviewer approves at the level you asked for or denies, optionally with a reason.
ICANN RDRS request	A request type of the RDRS kind sends your request to ICANN's Registration Data Request Service instead of querying RDAP, signed in with your own ICANN account. Jaddar assigns you a dedicated address (based on your name) that must be the email on that ICANN account, so ICANN and registrar correspondence lands in **RDRS Messages**; RDRS requests stay blocked until you confirm the registration, and you can only sign in to ICANN under that address.

A welcome bar shows the greeting, your display name, email, and group memberships. Below it sits the **RDAP/WHOIS Search** card, where all querying happens. The search form stays pinned to the top of the page as you scroll through a result.

Running a query

- **Query**: a domain, IP address, AS number, or entity handle. The type is detected from what you type.
- **Request Type**: click to open the selector, which has a **Search request types...** box and lists agreements grouped under each requestor group with a count. Each agreement row shows its name, the requestor group code, its description, and badges for every request type it offers (access level L0 to L3 plus 🌐 Standard, 🔒 Confidential, or ⚡ Exigent). Click an agreement with several request types to expand them, then pick one; an agreement with one request type is selected directly.
  - Rows marked **No credentials** or **Profile incomplete** are greyed out and cannot be picked; hover for the reason. A *Profile incomplete* row lists the missing fields with "Add to your profile in the Requestor Manager to use this agreement".
  - A **Contact:** line gives the data holder group's contact for the agreement: name, email, phone, and address.
  - After a JAKE search, **Filter by data holder response** offers a checkbox per TLD queried, such as *.com (2/5)* (two of your five agreements are honoured there) or *.org (none of yours)*. Ticking TLDs shows only the agreements honoured for them; **Clear filters** removes the filter.
  - Under the selector, the choice is summarised as *Level n* with its name (Public, Basic, Enhanced, Full), the request type, the requestor group code, and the **Contact:** line. The **Clear selection** icon empties it.
  - Warnings from Jadmin (an unreachable data holder group, a requestor group with no code) appear under the selector. If the list cannot be loaded, **Retry** asks again; "No request types available for your groups" means none of your groups has an active agreement.
- With nothing selected, "No request type selected — public RDAP lookup" appears beside the switches.
- **JAKE** asks the data holder to attach its JAKE data. **Conf.** and **Exig.** appear only when the selected request type allows confidential or exigent requests.
- **Parameters**: fields the request type defines appear in a panel beneath the form (text, numbers, dates, email, URL, lists, yes/no, JSON, and file uploads within the stated type and size). Required ones are starred and checked before the query is sent.
- Click **Search**. Each new search clears the notices left by the previous one. Errors open a dialog with the error code, query, source, server, and server response, and a **Try Again**

button. A file that does not meet the request type's requirements is reported as "Attachment Does Not Meet Requirements", with the reason per file.

Reading the result

A summary strip shows the time, the source badge (*iana* for a server found through the IANA bootstrap, *custom* for a data holder configured on the Data Holders page), the access level (L0 Minimal, L1 Basic, L2 Standard, L3 Full), the agreement, the object type, and the RDAP server that answered. **Parsed** and **JSON** switch between the rendered record and the raw response.

- The parsed view opens with an **Access:** bar giving the level and *Filtered by agreement*.
- When the data holder sends policy levels, a **Policy levels** bar names the policy, explains "S = sensitivity, V = validation. Hover a badge for detail.", and offers **Hide levels** / **Show levels**. Hovering a badge names the level, the policy fields it came from, and whether it applied or was superseded. Small L tags beside values give the access level each value requires.
- The record is grouped into collapsible sections: Contact Information, Basic Information, Important Dates, Status, DNS Information, Network Information, Links & References, Notices & Remarks, and Other Information. Click a heading to collapse or expand it. Redacted fields are counted and listed.

Two outcomes replace the record:

- **Manual Verification Required**: the request was submitted for review. The Request ID is shown, the page checks automatically every 5 seconds and shows the last check time, and **Check Now** checks at once. When the request is decided the result appears in place; if it lapses, "Request Expired" asks you to submit a new query.
- **Request Denied**: the data holder administrator refused the request. The reason, when supplied, and the Request ID are shown, with **New Search** to start again.

JAKE compliance block

With **JAKE** on, the result (or the pending or denied panel) is followed by a **JAKE Compliance** panel:

- **Data Holder Group Memberships**: each group with its status, then its **Templates** with *Max: Ln* and Confidential or Exigent markers, and each template's **Request Types** with level, 🔒 Confidential, ⚡ Exigent, and ✋ Manual markers, custom **Parameters**, and a summary of **RDAP Parameters** by category (Domain, Nameservers, Events, Registrant, Admin Contact, Tech Contact, Billing Contact, Registrar, DNSSEC, Network, ASN, Other). Each category is green when all its fields are enabled, amber for some, and red for none, with an enabled/total count.
- **Active Subscriptions**: subscription, template, requestor group code, access level, and request types.
- **Matched Subscription**: the agreement and access level used for this query.

A toast also reports how many valid agreements the data holder for that TLD returned, or that it returned none.

RDRS requests

Choosing an RDRS request type replaces the search form with a three-step flow, **Sign in**, **Request**, **Review & send**, with **Sign out of RDRS** at the right once you are signed in.

The first time, **Register with ICANN RDRS** appears instead and shows **Your Jaddar RDRS address**. Click **Copy**, click **Register at ICANN** and create your ICANN account with that exact address, then click **I've registered**. ICANN's verification email arrives in **RDRS Messages**.

1. **Sign in to ICANN RDRS**: the **ICANN email address** is fixed to your assigned address. Enter your **ICANN password** (the eye icon shows or hides it) and click **Sign in**; enter a **Verification code** and click **Verify** if ICANN asks for one, or **Cancel**. Your password is not stored. If ICANN rejects the sign-in, its own reason is shown; if ICANN later drops the session, you return to this step rather than being signed out of Jaddar.
2. **Request**:
   - **Domain**: enter the **Domain name** and click **Look up**. ICANN must identify the registrar of record before a request can be sent; the result names the registrar and IANA ID or explains why RDRS cannot take the domain (country-code and third-level domains, registrars outside RDRS, domains not found).
   - **Your contact details**: **Full name**, **Email address**, **Phone**, address lines, **City**, **State / province** (or **No state or province**), **ZIP / postal code** (or **No ZIP or postal code**), **Country code**, and **Organization**. Details from your ICANN account are prefilled; name and email taken from it are locked and marked "From your ICANN Account". A phone number must start with + and its country code and be a number that can actually be dialled; Jaddar checks it as ICANN does.
   - **Request details**: **Request category** (with **Describe your category**, 5–100 characters, for Other), **Priority** (Standard Request or Expedited Review Request, the latter needing a justification of 50–2000 characters), the voluntary confidentiality request, **Registration data requested** (grouped, with **Select all** / **Clear all**), **Countries where the data will be processed**, and **Describe your request** (50–2000 standard ASCII characters).
   - **Law enforcement**: answer whether a subpoena, court order, warrant, or other legal request has been issued; if yes, give **Response needed by** (a future date) and upload the **Law enforcement credential**.
   - **Legal basis & representation**: tick **I am asserting a legal basis for this request** to choose a **Legal basis** (GDPR Art. 6(1)a to f, or Other applicable law with 50–300 characters identifying it). Answer **Who are you submitting on behalf of?**; acting for a third party requires a **Power of attorney (or equivalent)**.

- **Supporting attachments**: up to 3 files. All attachments are PDF only, 5 MB maximum each.
- Any **Parameters** the request type defines follow; files uploaded there travel to ICANN as additional attachments.
- Click **Review request**; it stays disabled with "Look the domain up first." until the lookup succeeds.

3. **Review & send**: a summary of domain, registrar, requestor, category, priority, data requested, countries, description, and representation. Tick both acknowledgements and click **Send to ICANN**, or **Back** to edit. Jaddar repeats the lookup before sending, so the registrar cannot be changed in transit. If ICANN refuses the request, its reason is shown. On success, "Request sent to ICANN RDRS" confirms the domain and registrar; **Start another request** resets the form.

4.2 Requests

"RDAP Requests — Track your pending and completed RDAP data requests"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
RDRS	Registration Data Request Service (ICANN)
JSON	JavaScript Object Notation
PDF	Portable Document Format

CONCEPTS

Request history	Every search you run from Home is recorded here under your own account, whether it was answered, failed, or is waiting for review; RDRS requests are not listed. Searches sent with a file attachment are not recorded directly, though one held for review can appear once the list syncs with the data holder.
Pending status	A pending request's real status lives at the data holder. Loading the list, **Check Status**, and automatic polling ask the data holder and update the row; an approved request then gains its RDAP data at the level granted.
Automatic polling	Polling runs only while automatic polling is enabled, the interval is not Off, and you have at least one pending request. Your polling settings are saved to your account, not just this browser.
Cancellation	Only a pending request can be cancelled. Jaddar sends the cancellation and your reason to the data holder; if the data holder cannot be reached, the request is cancelled in Jaddar only and the data holder may still review it.
Remove from History	Removing a row deletes Jaddar's copy only; it does not cancel a pending request. A request the data holder still holds for you can reappear the next time the list syncs.

Statistics tiles: **Total Requests**, **Pending** (with a *Polling* marker while automatic polling runs), **Approved**, **Denied**, **Cancelled**. The toolbar carries **Compare**, **Export**, **Settings**, and **Refresh**.

Request History table

> **Filter:** by status (All Status, Pending, Approved, Denied, Cancelled, Error). Select rows individually or with **Select all**; the heading counts the selection and **Clear** empties it. Columns: Status, Type, Query, Agreements (a count, with the names on hover), Access Level (the level granted, or *Requested: n* while pending), Created, Resolved, Data, Actions.

- **View** (Data column, approved requests): opens the RDAP Data modal with **Parsed Data** and **Raw JSON** tabs and the requested and resolved times. Pending rows show *Waiting* and cancelled rows *Cancelled*.
- **Details** (Data column, denied requests): the query, when it was requested, the access level requested, the agreements involved, and the denial reason, or "No reason was provided by the data holder."
- **Check Status** (pending rows): asks the data holder now.
- **Cancel Request** (pending rows): with an optional **Cancellation Reason**; **Yes, Cancel Request** confirms and cannot be undone.
- **Remove from History**: confirmed first.

Compare

Select 2–4 approved requests that have RDAP data, then click **Compare (n)**. The modal lays the records out field by field, with a **Differences only** toggle and highlighted rows where the records disagree.

Export

Click **Export** (or **Export (n selected)**) to open **Export Options**:

- **Format**: Excel or PDF.
- **Rows to export**: **Selected rows** (when you have a selection), **Current page**, **A set number of rows** (counted from the newest request, regardless of the page you are on), or **All requests**. When the table is filtered by status, **Only include requests with status …** limits the export to it.
- **Columns to include**: Query Value, Query Type, Status, Access Level, Agreements, Data Holder, Created, Resolved, Error / Denial Reason, with **All** and **None** shortcuts. At least one is required.
- **Include the full RDAP response for approved requests**: adds a second worksheet in Excel, or a section after the summary table in PDF.
- A summary shows the resulting rows × columns; click **Export** to download.

Request Settings

- **Auto-Poll Interval**: a slider from Off to 1 hour, with quick buttons Off, 10s, 30s, 1min, 5min, 10min, 30min, and 1hr.
- **Enable automatic polling for pending requests**.
- **Notify on approval** and **Notify on denial**: marked as future features; they have no effect yet.
- **Save Settings** stores them.

4.3 Data Holders *(administrators only)*

"Manage custom RDAP data holders. IANA bootstrap is checked first; custom holders serve unmatched queries."

ABBREVIATIONS

RDAP	Registration Data Access Protocol
IANA	Internet Assigned Numbers Authority
TLD	Top-Level Domain
ASN	Autonomous System Number
AS	Autonomous System (a network identified by an ASN)
CIDR	Classless Inter-Domain Routing (IP range notation)
L0–L3	Access levels 0 (Public) to 3 (Full)

CONCEPTS

Data holder	An operator of an RDAP server holding registration data, usually running Jareg. Here you tell Jaddar where to reach each one and which TLDs, IP ranges, and AS number ranges it serves; anything no active data holder covers is looked up through the IANA bootstrap.
Resolution order	Domain and IP queries go to the data holders configured here first, then to those published by the look-aside registry when it is turned on (4.5), and to the IANA bootstrap last. AS number and entity queries try the IANA bootstrap first, then these data holders, then the registry's. If two active data holders cover the same TLD or range, only one of them is used, so keep coverage from overlapping.
Fallback	When a server cannot answer (not found, rate limited, a server error, a timeout, or no connection), Jaddar tries the next one in the order. A refusal, such as 401 or 403, is returned to the requestor as it stands.
Bearer token forwarding	With **Requires Authentication** on, Jaddar sends the requestor's Keycloak bearer token with each query, so the data holder can check it through token introspection and answer under an agreement. With it off, no token is sent, so the data holder can only answer as it would the public and refuses queries under an agreement. Jaddar also sends the requestor's email address and user ID to the configured data holders it queries, never to IANA-listed servers.

JAKE compliance view	The view asks the data holder itself for its JAKE data, which it builds from its connection to its group admin. A data holder whose group admin connection is not set up has nothing to report.

Click **Add Data Holder** to create one. Counters: **Total Holders**, **Active**, **TLDs Covered**, **IP/ASN Ranges**.

Resolution Tester

Choose **Domain**, **IP**, or **ASN**, enter a value, and click **Test** to see which RDAP server would handle the query first. The result reports the source, the server URLs, and *Auth Required* when the data holder needs a token, or states "No RDAP server found for" that value.

The table

Columns: Name, Base URL, Coverage, Auth, Status, Actions. The Status switch activates or deactivates a holder. Actions are **JAKE Compliance**, **Edit**, and **Delete** (confirmed; cannot be undone).

When the look-aside registry is turned on, the data holders it publishes follow yours on the last page, with a *Registry* badge in place of the status switch and *Managed in the registry* in place of the actions. They cannot be edited here.

Add or edit a data holder

- **Name** (required) and an optional **Description**.
- **Base URLs** (at least one required): RDAP server base URLs, each starting with *http://* or *https://*. **Add URL** adds another.
- **Coverage (at least one required)**: **TLDs** (e.g. com, net, tw) with **Add TLD**; **IP Ranges** in CIDR notation (e.g. 192.0.2.0/24, 2001:db8::/32) with **Add IP Range**; **ASN Ranges** as **Start** and **End** pairs (e.g. 64496–64511) with **Add ASN Range**.
- **Requires Authentication**: when on, choose **Forward Bearer Token**, the only method; the user's Keycloak bearer token is forwarded to this data holder.
- **Active**: whether the holder takes part in resolution.
- Click **Create** or **Update**.

JAKE Compliance view

Opened per data holder, this lists the group memberships with Active or Inactive status, the published templates with *Max: Ln* and Confidential or Exigent markers, their request types (level, 🔒 Confidential, ⚡ Exigent, ✋ Manual Approval, parameters, RDAP parameter counts), and **Active Subscriptions** with Subscriber, Template, and Access Level. **Refresh** reads it again. If nothing appears, the data holder's group admin connection is not configured.

4.4 Logs *(administrators only)*

"System Logs"

ABBREVIATIONS

RDAP	Registration Data Access Protocol
API	Application Programming Interface

CONCEPTS

Application logs	Jaddar keeps its own log in memory: the most recent 10,000 entries, lost when the backend restarts. **Clear** empties it permanently.
Keycloak events	This tab reads the login and token events Keycloak itself stores, including token introspections. **Clear** deletes them in Keycloak, not just from this view, and nothing is recorded unless saving events is turned on in Keycloak.

Two tabs: **Application Logs** and **Keycloak Events** (badged *Disabled* when Keycloak is not saving events). Statistics tiles: **Total Logs**, **Errors (24h)**, **Auth Events**, **Introspections**, **Unique Users**, **Unique Clients**. The header carries **Auto-refresh** and **Refresh**.

Application Logs

- Filters: **Search** over messages, **Level** (Debug, Info, Warning, Error, Critical), **Category** (Auth, RDAP, API, Admin, System, Introspection), and **Per Page**.
- Columns: Timestamp, Level, Category, Message, User/Client, IP, with **View details** per row.
- Pagination offers first, previous, next, and last page buttons plus a **Go to:** page box, and reports "Showing X - Y of Z entries".
- **Clear** deletes all application logs after confirmation. It cannot be undone.

Keycloak Events

- Filters: **Event Type** (Introspect Token, Introspect Token Error, Login, Login Error, Logout, Code to Token, Refresh Token, Client Login, Register, Token Exchange), **Client ID**, **User ID**, and **IP Address**, then **Search**.
- Columns: Timestamp, Event Type, User, Client, IP Address, Details.
- **Clear** removes all stored Keycloak events after confirmation.
- If the tab reports **Keycloak events are disabled.**, enable them in the Keycloak Admin Console under Realm Settings → Events → Login Events Settings → Save Events = ON.

4.5 Settings *(administrators only)*

"Deployment-wide settings for this Jaddar instance."

ABBREVIATIONS

RDRS	Registration Data Request Service (ICANN)
ICANN	Internet Corporation for Assigned Names and Numbers
IANA	Internet Assigned Numbers Authority

CONCEPTS

RDRS mail capture	Mail sent to Jaddar RDRS addresses is received through Mailgun, which keeps it only a few days. Jaddar collects new mail every couple of minutes into the matching requestor's **RDRS Messages** and ignores mail for addresses it never assigned. Until the server's Mailgun key is set, nothing is captured.
Retention	Captured messages are kept indefinitely unless you choose a period; with a period set, older messages are deleted each night and cannot be recovered. Requestors register their Jaddar address as their ICANN account email, so this is often their only copy of ICANN's correspondence.

The page has three tabs: **Logo**, **Look Aside Registry**, and **RDRS Mail**.

Logo

- **Upload logo** replaces the default logo on the sign-in screen and in the header. PNG, JPEG, GIF, or WebP, up to 2 MB; a wide image with a transparent background works best.
- The current logo is shown with its file name, size, who uploaded it, and when, labelled *Your logo* or *Default logo*.
- **Replace logo** swaps it; **Remove** reverts to the default.

Login notices

- Extra wording shown under the sign-in form: a privacy notice, an acceptable-use statement, a link to your terms. Nothing appears until you add an entry.
- Each notice has **Text** and an optional **Link (optional)**; a link turns the text into a hyperlink, a blank link leaves plain text.
- **Add notice** creates an entry; **Move up** / **Move down** set the display order; **Remove** deletes one. Up to 10 entries.
- **Save notices** commits the list.

Look Aside Registry

External Look Aside Registry lets Jaddar consult a shared directory of data holders in addition to the ones on the Data Holders page (4.3). "Locally configured data holders always win; the registry is checked next, and IANA remains the final fallback."

- The switch at the top turns the registry *Enabled* or *Disabled*. Turning it off stops Jaddar consulting it at once.

- **Registry URL**: the registry's feed directory. Jaddar reads *dataholders.json* from it.
- **Save** stores the settings and reads the registry straight away; if it cannot be read, the settings are still saved and the error is shown. **Refresh now** reads it again.
- While enabled, a banner shows the result of the last read, when it was checked, and when the registry last published its feed.
- **Data holders from the registry ({n})** expands to list each one with its Name, Base URLs, Coverage, and Authentication (*Bearer token forwarded* or *None*). "These are published by the registry and cannot be edited here. To change one, change it in the registry."
- Only administrators can change these settings.

RDRS Mail

- **RDRS Mail Capture** explains where the mail comes from. If mail capture is not configured on the server, a warning says so.
- **Keep captured messages for**: 7 days, 30 days, 90 days, 6 months, 1 year, or Indefinitely. Click **Save** to apply. The note beneath states the nightly deletion or, for Indefinitely, why keeping everything matters.
- **Stored messages**, **Last capture run**, and **Last result** report the state of capture.
- **Capture now** collects new mail at once. **Apply retention now** deletes messages older than the retention period straight away; it is disabled while retention is Indefinitely.

5. Protocols and Standards

This part lists the protocols and standards the four applications and their helper services use, what each one is used for, and where the suite departs from a standard. Each entry names the specification where the suite follows one. Names in *italics* are server configuration settings or files rather than screens.

5.1 Registration data (RDAP)

ABBREVIATIONS

RDAP	Registration Data Access Protocol
IANA	Internet Assigned Numbers Authority
AS	Autonomous System (a network identified by an ASN)
SHA-256	Secure Hash Algorithm, 256-bit
RFC	Request for Comments (an IETF standard)
HTTP, HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
EPP	Extensible Provisioning Protocol
ISO	International Organization for Standardization
UTC	Coordinated Universal Time
URI	Uniform Resource Identifier

STANDARDS

RDAP over HTTPS (RFC 7480, RFC 9082)	Jareg answers lookups at **/domain/{name}**, **/ip/{address}** and **/autnum/{number}** with the **application/rdap+json** media type. Jaddar sends lookups to data holders and public RDAP servers at **/domain/**, **/ip/**, **/autnum/** and **/entity/**, asking for **application/rdap+json**. Nameserver, entity and help lookups and RDAP searches are not served by Jareg.
RDAP responses (RFC 9083)	Jareg's answers carry the standard members: **objectClassName**, **ldhName** and **unicodeName**, **status**, **events** (ISO 8601 with a UTC offset), **entities**, **nameservers**, **secureDNS** (**delegationSigned**, **dsData**, **keyData**), **links**, **notices**, **remarks**, **port43** and **publicIds**. Errors use **errorCode**, **title** and **description**.
rdapConformance	Every Jareg answer declares **rdap_level_0**, **icann_rdap_response_profile_0** and **icann_rdap_technical_implementation_guide_0**. The suite's own additions (below) are not registered as RDAP extensions.
Contacts as jCard (RFC 7095)	Contacts are sent in **vcardArray** as vCard 4.0 with **fn**, **org**, **email**, **tel** and **adr**. Telephone numbers are **tel:** URIs (RFC 3966) typed voice or fax. Imported records hold numbers in the EPP form **+CC.number**, such as +1.5551234567.

Status values	Domain status values are EPP status codes in their RDAP spelling, such as "client transfer prohibited".
AS numbers (RFC 6793)	AS numbers up to 4294967295 (32-bit) are accepted throughout.
Bootstrap (RFC 9224, RFC 8521)	Jaddar finds the authoritative RDAP server from IANA's bootstrap files (**dns.json**, **ipv4.json**, **ipv6.json**, **asn.json**, and **object-tags.json** for entity handles) at data.iana.org, cached for an hour with the last good copy kept as a fallback. Domain and IP queries try the data holders configured in Jaddar first, then the look-aside registry, then IANA; AS number and entity queries try IANA first (4.3).
Look-aside registry / common repository	A shared directory that sits beside the IANA bootstrap, called the look-aside registry in Jaddar and the common repository in Jadmin and Jaremin. Jaddar reads its **dataholders.json** when the registry is turned on (4.5) and caches it for five minutes. Jadmin reads its **group-registries.json** to add data holder groups (1.6). Jaremin announces its globally published groups and templates to it every 15 minutes (2.11), presenting a bearer token the repository issued on the first announcement.
Redaction	Jareg removes a field the request may not see, or replaces its value with "REDACTED", and adds a notice titled **Redacted Fields** of type **object redacted due to authorization** listing the paths removed. It does not send the RFC 9537 **redacted** member. Jaddar can display an RFC 9537 **redacted** array when another RDAP server sends one.
Agreement query parameters	Beyond the RDAP path, Jaddar adds **dataHolderGroup**, **requestorGroup** and **requestType** (the request type's code), the switches **jakeCompliance**, **confidential** and **exigent**, and **customParams** (the request type's parameters as one JSON value). The requestor's token travels as **Authorization: Bearer**, with **X-Requestor-Email** and **X-Requestor-Sub** headers.
Request type and agreement codes	Opaque 16-character hexadecimal codes, the first 16 characters of a SHA-256 hash, assigned by Jaremin. Jaddar refuses a request type code in any other form. Agreement codes appear only in the JAKE block.
Held requests	A query that needs manual review is answered with HTTP **202 Accepted** and **status** "pending", a **requestId**, a **pollUrl** (**/api/rdap/status/{id}**) and **expiresAt**. This exchange is specific to the suite; RDAP itself defines none.
JAKE block	With **jakeCompliance=true** the answer gains a top-level **jakeCompliance** member, specific to the suite, listing the data holder's groups, their templates and request type codes, the active subscriptions and the agreement codes it honours (3.1, 4.1).
Public Information Services	A standard RFC 9083 notice with that title whose **links** are the joining links of requestor groups the data holder serves (1.3). Jareg reads them from the **joining_url** claim in introspection answers and caches them for five minutes.

5.2 Sign-in and tokens (OAuth 2.0, OpenID Connect, JWT)

ABBREVIATIONS

JWT	JSON Web Token
RFC	Request for Comments (an IETF standard)
HTTP, HTTPS	Hypertext Transfer Protocol (Secure)

API	Application Programming Interface
JWKS	JSON Web Key Set

STANDARDS

Keycloak	The identity service for the requestor side. Jadmin and Jaddar users, their requestor groups (as Keycloak groups) and their roles (**jaddar_master_admin**, **group_admin**, **requestor_group_admin**, **requestor_group_user**) live there. Jadmin manages them through the Keycloak Admin REST API. By default an idle Keycloak session ends after 15 minutes, a session lasts at most 10 hours, and an access token lives 5 minutes. Brute-force protection and user and admin events are on; Jaddar reads the events for its Logs page (4.4).
OAuth 2.0 grants (RFC 6749)	Jaddar and Jadmin sign users in on the server with the resource owner password grant, scope **openid profile email**, so the browser never holds a token; refresh tokens keep the session going. Service-to-service tokens use the client credentials grant, which is the only grant Jadmin's own token endpoint (**/api/v1/external/oauth/token**) accepts.
Bearer tokens (RFC 6750)	Every call made for a signed-in requestor carries **Authorization: Bearer** with the Keycloak access token.
JWT and JWKS (RFC 7519, RFC 7517)	Keycloak's access tokens are signed JWTs. Jaddar checks them against Keycloak's published keys (its **/protocol/openid-connect/certs** endpoint), their validity times, and that they were issued to or for the Jaddar client. Jadmin does not read tokens itself: it asks Keycloak about each one through introspection. Jaremin and Jareg issue their own HS256-signed JWTs to their local administrator accounts (8 hours for Jaremin and 24 hours for Jareg, with 7-day refresh tokens).
OpenID Connect Discovery	Jaddar reads Keycloak's **/.well-known/openid-configuration** for its endpoints and serves its own copy at the same path. Jadmin reads it to find the **introspection_endpoint** that Keycloak advertises publicly, which it gives to requestor groups and data holders (1.3). Signing out uses the **end_session_endpoint**.
Token introspection (RFC 7662)	How a data holder confirms a requestor's token on every query, with no caching. Jareg posts the token, form-encoded, to the subscription's introspection URL with HTTP Basic client authentication using the introspection credentials Jadmin delivered. An inactive token answers only **active: false**. The answer must bind the token to the requestor group through its **groups**, **resource_access** or **aud** claims. Jadmin's introspection endpoint (**/api/v1/external/oauth/token/introspect**) adds **user_fields** and **missing_user_fields** with the member information the agreement requires.
Introspection clients	For each active subscription Jadmin creates a confidential Keycloak client named **sub-{request ID}-introspect** that can do nothing but introspect. The requestor group is given that client's **introspect** role, which places the client in the audience of its members' tokens; without it Keycloak refuses with "client not in audience". A claim mapper adds the group's **joining_url** to introspection answers only.

5.3 Connections between the applications

ABBREVIATIONS

RDRS	Registration Data Request Service (ICANN)
SHA-256	Secure Hash Algorithm, 256-bit
RFC	Request for Comments (an IETF standard)
HTTP, HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation

STANDARDS

Client credentials over HTTP Basic (RFC 7617)	Each Client ID and Client Secret pair that Jaremin issues (Overview) is sent as **Authorization: Basic**. Jadmin sends the subscription's own pair on calls about one subscription and the Group Admin Credentials otherwise, with **X-Requestor-Service: requestor-manager**. Jareg sends its data holder pair on calls to **/api/external/my/...**. Jaremin keeps only a BCrypt hash of each secret, which is why a lost secret can be replaced but not recovered.
HTTP Message Signatures (RFC 9421)	Jadmin signs its calls about a subscription with that subscription's Ed25519 key (RFC 8032), and Jaremin checks them against the registered public key (2.2). The signature covers the method, the path and the body digest, sent as **Signature-Input: sig1=("@method" "@path" "content-digest");created={time};keyid="{client ID}"** and **Signature: sig1=:{signature}:**. Public keys are X.509 SubjectPublicKeyInfo and private keys PKCS #8, both in Base64.
Content-Digest (RFC 9530)	A SHA-256 digest of the exact body, sent as **Content-Digest: sha-256=:{digest}:** and covered by the signature, so a signed body cannot be altered.
Signature timing	A signature is accepted for 300 seconds either side of its **created** time (*peer.signatures.max-skew-seconds*). Only a signed call can carry member information.
Jaddar to Jadmin	Jaddar fetches a user's agreements with the user's bearer token and **X-Requestor-Agent-Id** (normally JADDAR-001).
Helper services	The provisioning sidecar (Instances, 2.8), the document scanner (3.10) and the RDRS sidecar (4.1) accept calls only with a shared secret in **X-Provisioner-Secret**, **X-Scanner-Secret** or **X-Rdrs-Secret**, compared in constant time. They listen on the server's loopback address only.
Data formats	Calls between the applications carry JSON (RFC 8259). File attachments travel as multipart/form-data (RFC 7578).

5.4 Transport and web security

ABBREVIATIONS

RDRS	Registration Data Request Service (ICANN)
SHA-256	Secure Hash Algorithm, 256-bit

RFC	Request for Comments (an IETF standard)
HTTP, HTTPS	Hypertext Transfer Protocol (Secure)
TLS	Transport Layer Security
ACME	Automatic Certificate Management Environment
CSRF	Cross-Site Request Forgery
CORS	Cross-Origin Resource Sharing
AES-256-GCM	Advanced Encryption Standard, 256-bit key, Galois/Counter Mode

STANDARDS

HTTPS and TLS	The nginx front end redirects HTTP to HTTPS, serves HTTP/2 with Let's Encrypt certificates, and passes **X-Real-IP**, **X-Forwarded-For**, **X-Forwarded-Proto** and, where an application sits under a path, **X-Forwarded-Host**, **X-Forwarded-Port** and **X-Forwarded-Prefix**. Keycloak runs behind it with forwarded headers trusted.
ACME (RFC 8555)	When Jaremin creates an instance (2.8), the provisioning sidecar requests its certificate from Let's Encrypt with certbot's nginx plugin, then tests and reloads nginx. A failure here does not stop the instance, because a wildcard certificate may already cover the name.
Internal TLS	Inside the deployment, a private certificate authority created by *scripts/generate-infra-certs.sh* (RSA 4096-bit, SHA-256) signs the server certificates. Backends trust that authority alongside the usual public ones, and reach Keycloak over HTTPS for sign-in, administration and introspection. The helper services can also require client certificates (mutual TLS) when *MTLS_ENABLED* is set; it is off by default.
Browser sessions	Each web application keeps the user's tokens on its server, encrypted with AES-256-GCM, and gives the browser only an httpOnly session cookie, marked Secure in production and SameSite=Lax. Sessions end after 15 minutes idle (*SESSION_IDLE_MINUTES*) and 8 hours in all (*SESSION_MAX_HOURS*), with the warning 120 seconds before. While you are active, the page sends a heartbeat about every two minutes, which also renews a Keycloak token about to expire; activity is shared between open tabs.
CSRF protection	Sign-in and other form submissions in the web applications carry a token tied to the session and are refused with 403 when it does not match.
Content Security Policy	Pages are served with a strict policy: scripts only from the application itself with a fresh nonce per page, no plug-ins, and no framing by other sites.
Other response headers	Backends send **X-Content-Type-Options: nosniff**, **X-Frame-Options: DENY**, **Referrer-Policy: strict-origin-when-cross-origin**, a **Permissions-Policy** that turns off camera, microphone and location, and **Cache-Control: no-store**.
CORS	Jaddar, Jaremin and Jareg accept browser calls only from the origins listed in their configuration.
Rate limits	Jaddar accepts up to 240 requests a minute from each client address, taken from **X-Forwarded-For** or **X-Real-IP**.
Updates on screen	Pages that change on their own, such as pending requests, subscriptions, dashboards and the RDRS mailbox, poll the server at intervals. The suite uses no WebSockets or server-sent events.

5.5 ICANN RDRS and email

ABBREVIATIONS

RDRS	Registration Data Request Service (ICANN)
ICANN	Internet Corporation for Assigned Names and Numbers
HTTP, HTTPS	Hypertext Transfer Protocol (Secure)
API	Application Programming Interface
SAML	Security Assertion Markup Language
HMAC	Hash-based Message Authentication Code
ITU-T	International Telecommunication Union, Telecommunication Standardization Sector

STANDARDS

RDRS sign-in	ICANN's Registration Data Request Service offers only a browser sign-in, so the RDRS sidecar drives a headless browser through it: ICANN's account page, ICANN's Okta identity service (which uses SAML internally), any verification code ICANN asks for, and finally an OAuth 2.0 implicit-flow redirect that returns an access token. Settings come from ICANN's **/api/config**. Your ICANN password is used for that sign-in only and is not stored.
RDRS API	Jaddar exchanges the access token at ICANN's **/users/authenticate** for an **accessToken** cookie that it keeps on its server and never passes to the browser. It then calls **/users/profile**, **/requests/lookup/{domain}** and **/requests/create** with that cookie.
Telephone numbers (ITU-T E.164)	An RDRS phone number must start with + and its country code and be a number that can be dialled, checked with Google's libphonenumber as ICANN does. It is sent in international format.
Outgoing email	Password resets and notifications from every application are sent through the SendGrid v3 Web API over HTTPS. The suite does not speak SMTP itself.
Incoming RDRS mail	Mail to requestors' Jaddar RDRS addresses is received by Mailgun. Jaddar collects it from Mailgun's events API every two minutes and through Mailgun's inbound webhook, which it accepts only with a valid HMAC-SHA256 signature less than five minutes old, ignoring messages it has already stored.

5.6 Data stores, files and formats

ABBREVIATIONS

RDAP	Registration Data Access Protocol
ICANN	Internet Corporation for Assigned Names and Numbers
JDBC	Java Database Connectivity
OCR	Optical Character Recognition
DPI	Dots Per Inch

SHA-256	Secure Hash Algorithm, 256-bit
RFC	Request for Comments (an IETF standard)
TLS	Transport Layer Security
JSON	JavaScript Object Notation
API	Application Programming Interface
AES-256-GCM	Advanced Encryption Standard, 256-bit key, Galois/Counter Mode
ISO	International Organization for Standardization
UTC	Coordinated Universal Time
URI	Uniform Resource Identifier
CSV	Comma-Separated Values
PDF	Portable Document Format
XLSX, DOCX	Excel and Word files (Office Open XML)

STANDARDS

PostgreSQL	Each application has its own PostgreSQL 15 database, as does Keycloak and each hosted instance. Connections use TLS and verify the server against the deployment's certificate authority (**sslmode=verify-ca**). Database structure is versioned with Flyway migrations.
MariaDB	The sample RDAP database runs on MariaDB and accepts encrypted connections only.
External Database (JDBC)	Jareg's External Database (3.9) connects through JDBC to PostgreSQL (**jdbc:postgresql:**) or MariaDB (**jdbc:mariadb:**). JDBC URL options that can run code or read local files are refused. To encrypt the connection, add the driver's TLS options to the JDBC URL.
Stored secrets	Secrets kept in the databases, such as client secrets and connection passwords, are encrypted with AES-256-GCM. Local account passwords are stored as BCrypt hashes, and password-reset links are stored only as SHA-256 hashes and expire after 60 minutes.
Docker Engine API	The provisioning sidecar creates, starts, stops, updates and removes hosted instances (2.8) through the Docker Engine API on the server's Docker socket, and writes their nginx configuration.
File attachments	Jareg accepts PDF, DOCX, XLSX, TXT, JPEG and PNG attachments, checks each file's leading bytes against its claimed type, rejects PDFs containing scripts or automatic actions, and records a SHA-256 hash. File Automation (3.10) applies its rules to these attachments; it is not a file-transfer protocol, and the suite uses no FTP, SFTP or shared-folder transfer.
Document checks and OCR	The document scanner reads PDFs with PyMuPDF and runs Tesseract OCR at 300 DPI on pages with almost no text, up to 10 pages. It reads DOCX text without OCR, and image size, resolution and text with Pillow and Tesseract.
Imports and exports	Jareg imports RDAP records as CSV or JSON (3.4). Jaddar exports results as XLSX (Office Open XML) or PDF (4.2).

Release feed	Jaddar's release notes are an Atom 1.0 feed (RFC 4287) at **/versions.atom**, with RFC 3339 timestamps and tag URIs (RFC 4151) as entry IDs.
API description	Jadmin describes its API in OpenAPI 3 at **/v3/api-docs**, browsable at **/swagger-ui.html**.
Countries, dates and times	Country pickers list ISO 3166-1 countries grouped into ICANN's five regions (Africa, Asia-Pacific, Europe, Latin America and the Caribbean, North America); the country name is stored. Dates and times are exchanged in ISO 8601 with a UTC offset.

5.7 Connections at a glance

ABBREVIATIONS

RDAP	Registration Data Access Protocol
RDRS	Registration Data Request Service (ICANN)
ICANN	Internet Corporation for Assigned Names and Numbers
IANA	Internet Assigned Numbers Authority
RFC	Request for Comments (an IETF standard)
HTTP, HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
API	Application Programming Interface
ACME	Automatic Certificate Management Environment

STANDARDS

Jaddar → Keycloak	OpenID Connect: password and refresh grants, discovery, key set, end session.
Jaddar → data holders and RDAP servers	RDAP over HTTPS, with the requestor's bearer token and the agreement query parameters (5.1).
Jaddar → IANA	RFC 9224 bootstrap files over HTTPS.
Jaddar → look-aside registry	Its **dataholders.json** list of data holders over HTTPS.
Jaddar → Jadmin	HTTPS and JSON with the user's bearer token, to list agreements.
Jaddar → RDRS sidecar → ICANN	Shared-secret header, then ICANN's browser sign-in and the RDRS API (5.5).
Jadmin → Keycloak	Introspection of every user token, the Admin REST API for users, groups and introspection clients, and discovery.
Jadmin → Jaremin	HTTPS and JSON with HTTP Basic client credentials, signed with the subscription's Ed25519 key on calls about a subscription (5.3).
Jadmin → common repository	Its **group-registries.json** listing of data holder groups over HTTPS.

Jaremin → Jareg	HTTPS call asking a data holder to run a subscription's retrieval test against its test data (2.2).
Jaremin → common repository	Announcement every 15 minutes with the repository's bearer token.
Jaremin → provisioning sidecar	Shared-secret header; the sidecar then uses the Docker Engine API, nginx and ACME (5.4, 5.6).
Jareg → Jaremin	HTTPS and JSON with HTTP Basic data holder credentials, to read its groups, agreements and the subscriptions it must honour.
Jareg → introspection URL	RFC 7662 token introspection with the subscription's introspection credentials, on every query under an agreement.
Jareg → document scanner	Shared-secret header, multipart file and its format requirements.
Applications → SendGrid; Mailgun → Jaddar	Outgoing email over the SendGrid Web API; incoming RDRS mail by Mailgun webhook and events API.